



LABORATORIO DE PROGRAMACIÓN VIRTUAL
PARA EL DISEÑO LÓGICO DIGITAL
MEDIANTE LÓGICA RECONFIGURABLE

Manual de Usuario

Treviño Hernández Jonathan
Peñarrieta Villa Jesús

Versión 1.0 – 2026

Escuela Superior de Cómputo
Ingeniería en Sistemas Computacionales

Manual de Usuario

Laboratorio de Programación Virtual para el Diseño Lógico Digital
mediante Lógica Reconfigurable

Treviño Hernández Jonathan – Peñarrieta Villa Jesús

2026

Este documento fue elaborado con fines académicos.
Plataforma: Moodle + Virtual Programming Lab (VPL)

Índice general

Prefacio	vii
I Guía del Alumno	1
1 Acceder a la Actividad VPL	3
1.1 Requisitos previos	3
1.2 Ingreso a la plataforma Moodle	3
1.3 Localizar y abrir la actividad VPL	3
2 Entrar al Editor de VPL	7
2.1 Descripción del entorno de edición	7
2.2 Ajustes del editor	8
3 Crear Archivos VHDL	9
3.1 Estructura básica de un archivo VHDL	9
3.2 Crear un nuevo archivo en VPL	9
3.3 Nomenclatura y organización de archivos	11
4 Compilar Archivos VHDL	13
4.1 Proceso de compilación en VPL	13
4.2 Interpretación de errores y advertencias	14
4.3 Buenas prácticas de compilación	15
5 Generación de Código Testbench	17
5.1 ¿Qué es un testbench?	17
5.2 Generación automática en el entorno VPL	17
5.3 Personalización del testbench generado	19
6 Uso de la Simulación en VPL	21
6.1 Iniciar la simulación	21
6.2 Visualización de formas de onda	23
6.3 Depuración mediante simulación	25
7 Uso de la Evaluación Automática	27
7.1 ¿Cómo funciona la evaluación automática?	27
7.2 Enviar el diseño para evaluación	27
7.3 Interpretar la retroalimentación	28
7.4 Reintentos y límite de envíos	28

II	Guía del Profesor	31
8	Creación de Actividad VPL	33
8.1	Precondiciones	33
8.2	Crear una actividad VPL	33
8.2.1	Configuraciones generales	35
8.2.2	Restricciones de entrega	36
8.2.3	Otras configuraciones	37
9	Acciones Permitidas	39
9.1	Tipos de acciones en VPL	39
9.2	Configurar acciones disponibles para el alumno	39
10	Configuración de Casos de Prueba	43
10.1	Archivo <code>vpl_evaluate.cases</code>	43
10.1.1	Sintaxis general de los casos de prueba	43
10.1.2	Sintaxis específica para evaluar código VHDL	45
10.2	Usar el archivo de casos para evaluar diseños VHDL	47
11	Test de Evaluación por Parte del Profesor	51
11.1	Propósito	51
11.2	Estructura de la carpeta de pruebas	51
11.2.1	Ejemplos de nombres de subcarpeta	52
11.2.2	Estructura de directorios	52
11.2.3	Actividades con variaciones	53
11.3	Pasos para ejecutar el test	53
11.4	Reporte de evaluación	54
11.4.1	Configuración previa de los casos de prueba	54
11.4.2	Reporte de compilación	54
11.4.3	Reporte de evaluación (resumen)	55
11.4.4	Reporte de evaluación completo (<i>Full Evaluation Report</i>) . . .	55
11.4.5	Reporte con casos fallidos	56
A	Glosario de Términos	57
B	Referencia Rápida de Comandos	59
C	Recursos y Referencias	61

Índice de figuras

1.1	Vista de cursos disponibles en Moodle para el alumno	4
1.2	Página de la actividad VPL con información de entrega	5
2.1	Editor de VPL con el diálogo para crear un nuevo archivo	8
3.1	Archivo <code>alu.vhd</code> creado y visible en el editor de VPL	10
3.2	Código VHDL escrito en el archivo <code>alu.vhd</code>	10
4.1	Resultado de compilación exitosa en el panel de resultados	14
4.2	Vista en pantalla completa del editor con compilación exitosa	14
4.3	Panel de resultados mostrando errores de compilación	15
5.1	Error al intentar generar testbench desde un archivo VHDL vacío	18
5.2	Testbench generado automáticamente para el diseño VHDL	19
5.3	Testbench con vectores de prueba añadidos manualmente	20
6.1	Salida detallada del proceso de compilación al ejecutar Debug	22
6.2	GTKWave abierto con la jerarquía de señales y área de visualización vacía	22
6.3	Árbol de señales SST expandido con menú contextual para importar señales	23
6.4	Diálogo de confirmación para importar todas las señales del componente	24
6.5	Formas de onda de las señales del diseño visualizadas en GTKWave	24
6.6	Señales vectoriales expandidas mostrando cada bit individualmente en GTKWave	25
6.7	Directiva <code>vpl_stop_time</code> declarada en el testbench y confirmada por el compilador	26
6.8	GTKWave con el eje temporal ajustado al tiempo definido por <code>vpl_stop_time</code>	26
7.1	Vista de entrega en Moodle con fecha de envío y archivo registrado	28
8.1	Cursos disponibles en Moodle	34
8.2	Cursos donde se desea agregar la actividad VPL	34
8.3	Crear actividad VPL	35
8.4	Configuraciones generales de la actividad VPL	36
8.5	Restricciones de entrega en la actividad VPL	37
8.6	Guardar cambios en la actividad VPL	38
8.7	Visión general de la actividad VPL	38
9.1	Mas configuraciones de la actividad VPL	40
9.2	Opciones de ejecución en la actividad VPL	41

9.3	Permitir opciones de ejecución en la actividad VPL	41
10.1	Archivo de configuración de casos de prueba en la actividad VPL . .	48
10.2	Archivo de casos de prueba creado y guardado en la actividad VPL .	48
11.1	Reporte generado por VPL al evaluar todos los casos de prueba correctamente	54
11.2	Detalle del <i>Evaluation output</i> y resumen estadístico al final del reporte	55
11.3	Reporte de evaluación con casos fallidos detectados por el evaluador automático	56

Índice de cuadros

10.1 Parámetros de configuración global	43
10.2 Parámetros por caso de prueba	44

Prefacio

Este manual describe el uso del **Laboratorio de Programación Virtual (VPL)** integrado en *Moodle* para el diseño y verificación de circuitos digitales escritos en VHDL mediante lógica reconfigurable.

El documento se divide en dos partes independientes:

- **Parte I — Guía del Alumno:** acceso, edición, compilación, simulación y evaluación automática de diseños VHDL dentro del entorno VPL.
- **Parte II — Guía del Profesor:** creación y configuración de actividades VPL, definición de casos de prueba, generación de *testbenches* y análisis de reportes de evaluación.

Sobre este documento

Se recomienda leer la documentación oficial de VPL for Moodle para saber más sobre sus características avanzadas, personalización y solución de problemas comunes: <https://vpl.dis.ulpgc.es/documentation/vpl-4.4.2/>

Parte I

Guía del Alumno

Acceder a la Actividad VPL

1.1 Requisitos previos

Para utilizar el laboratorio virtual VPL es necesario contar con:

- **Navegador web actualizado:** se recomienda Chrome, Firefox, Brave o Edge.
- **Cuenta activa en Moodle:** el alumno debe tener usuario y contraseña válidos para iniciar sesión en la plataforma.
- **Matrícula en el curso:** la cuenta debe estar inscrita en el curso que contiene la actividad VPL.
- **Conexión a internet estable:** la compilación y simulación se ejecutan en el servidor; una conexión inestable puede interrumpir la ejecución.

1.2 Ingreso a la plataforma Moodle

Paso 1

Abrir el navegador e ingresar la dirección URL de la plataforma *Moodle* proporcionada por el profesor o institución.

Paso 2

En la página de inicio de sesión, introducir el **nombre de usuario** y la **contraseña** y hacer clic en **Iniciar sesión**.

1.3 Localizar y abrir la actividad VPL

Una vez dentro de *Moodle*, la sección **My courses** muestra los cursos en los que el alumno está inscrito.

Paso 1

Hacer clic en **My courses** en la barra de navegación superior para ver el listado de cursos disponibles.

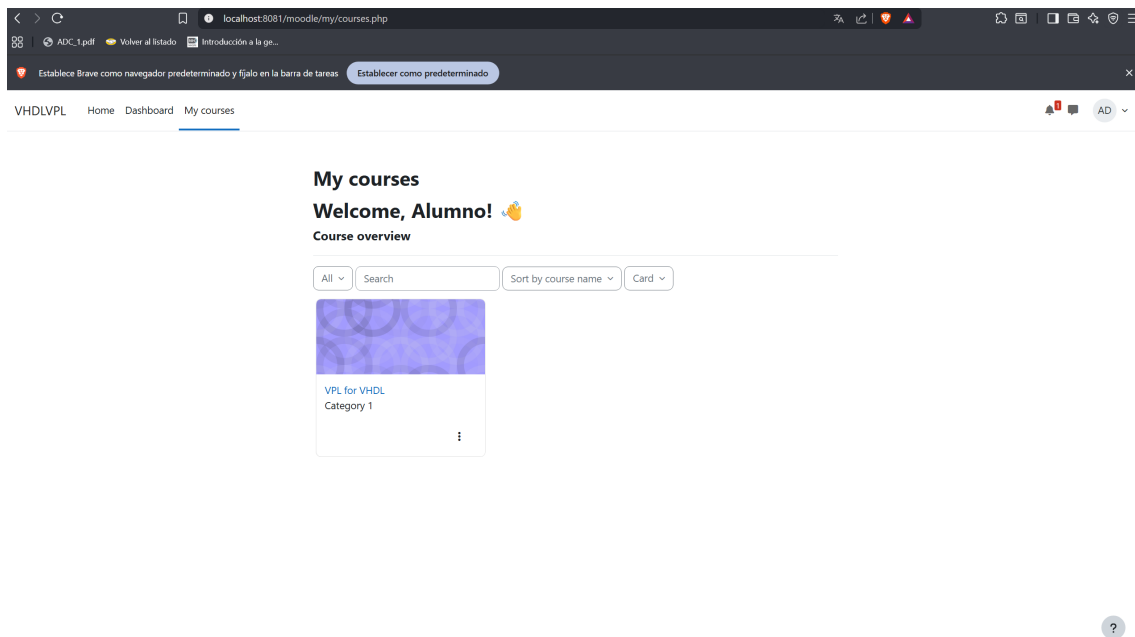


Figura 1.1: Vista de cursos disponibles en Moodle para el alumno

Paso 2

Hacer clic sobre la tarjeta del curso correspondiente para acceder a él.

Paso 3

Dentro del curso, localizar la actividad VPL en la lista de actividades y hacer clic sobre ella para abrir su página.

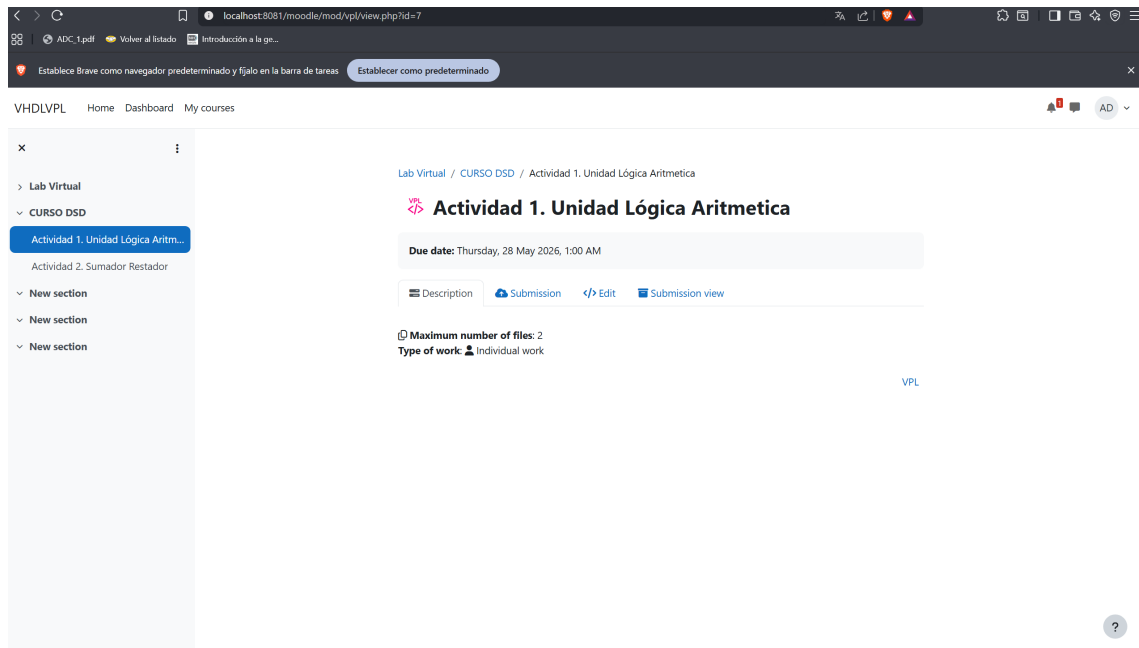


Figura 1.2: Página de la actividad VPL con información de entrega

En la página de la actividad (Figura 1.2) se muestra información relevante:

- **Due date:** fecha y hora límite de entrega.
- **Maximum number of files:** número máximo de archivos permitidos (generalmente el archivo VHDL fuente y su testbench).
- **Type of work:** modalidad de la actividad (individual o grupal).

Para ingresar al editor y comenzar a trabajar, hacer clic en el botón **VPL** en la parte inferior de la página o en la pestaña **Edit**.

Entrar al Editor de VPL

2.1 Descripción del entorno de edición

Al acceder al editor de VPL se carga un entorno compuesto por tres áreas principales:

- **Barra de herramientas:** en la parte superior contiene las acciones disponibles: crear archivo, guardar, ejecutar (*Run*), depurar (*Debug*), generar testbench y evaluar, entre otras.
- **Panel de archivos y editor:** área central donde se escribe y edita el código VHDL. Las pestañas superiores muestran los archivos abiertos y el panel *File list* los lista a la izquierda.
- **Panel de resultados (*Compilation*):** aparece a la derecha al ejecutar cualquier acción y muestra la salida del compilador o del simulador.

Para crear el primer archivo de trabajo, hacer clic en el ícono **Crear nuevo archivo** (primer ícono de la barra de herramientas). Aparece un cuadro de diálogo solicitando el nombre del nuevo archivo (Figura 2.1). Al acceder por primera vez al editor, en automático lanza el recuadro para solicitar el nombre del nuevo archivo, por lo que el paso de hacer clic en el ícono de crear nuevo archivo se puede omitir si es la primera vez que se accede al editor.

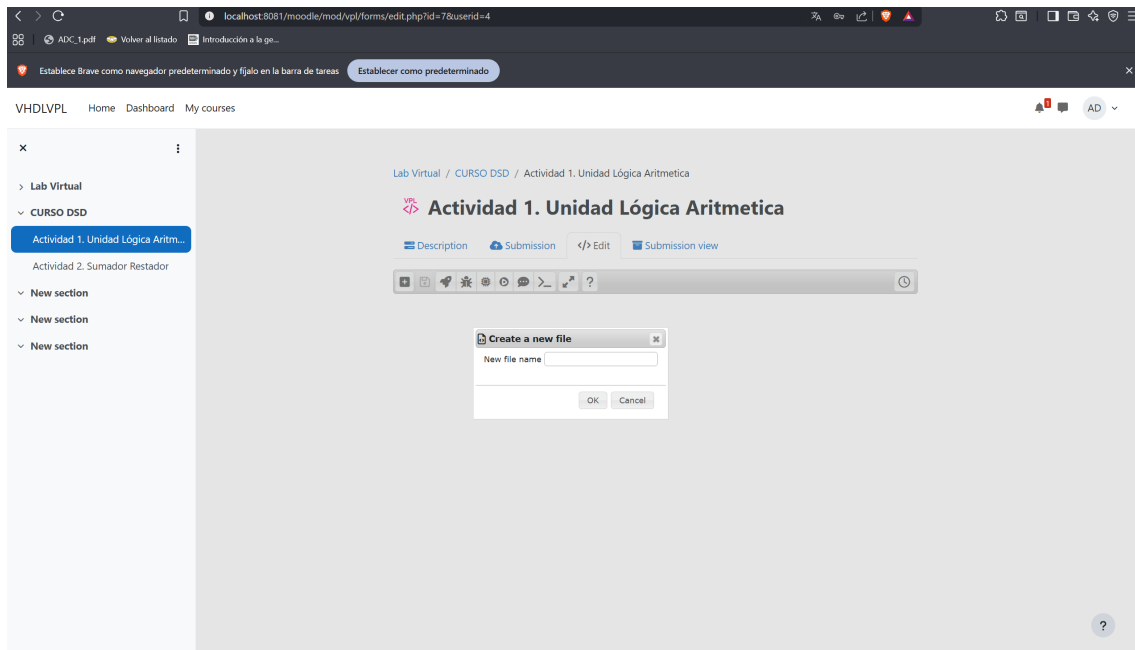


Figura 2.1: Editor de VPL con el diálogo para crear un nuevo archivo

2.2 Ajustes del editor

El editor incluye resaltado de sintaxis para VHDL, indentación automática y deshacer/rehacer cambios con los controles de la barra de herramientas.

Pantalla completa

El ícono de pantalla completa (extremo derecho de la barra de herramientas) amplía el editor para aprovechar todo el espacio disponible, lo que resulta útil al revisar el código y los resultados al mismo tiempo.

Crear Archivos VHDL

3.1 Estructura básica de un archivo VHDL

Un archivo VHDL mínimo consta de tres partes:

1. **Declaración de bibliotecas** — importación de paquetes estándar necesarios (por ejemplo `ieee.std_logic_1164`).
2. **Entidad (*entity*)** — define el nombre del componente y sus puertos de entrada y salida.
3. **Arquitectura (*architecture*)** — describe el comportamiento o la estructura interna del componente.

3.2 Crear un nuevo archivo en VPL

Paso 1

En la barra de herramientas, hacer clic en el ícono **Crear nuevo archivo**.

Paso 2

En el cuadro de diálogo, escribir el nombre del archivo incluyendo la extensión `.vhd` (por ejemplo: `alu.vhd`) y hacer clic en **OK**.

Paso 3

El archivo recién creado aparece como una nueva pestaña en el editor y queda listado en el panel *File list* (Figura 3.1).

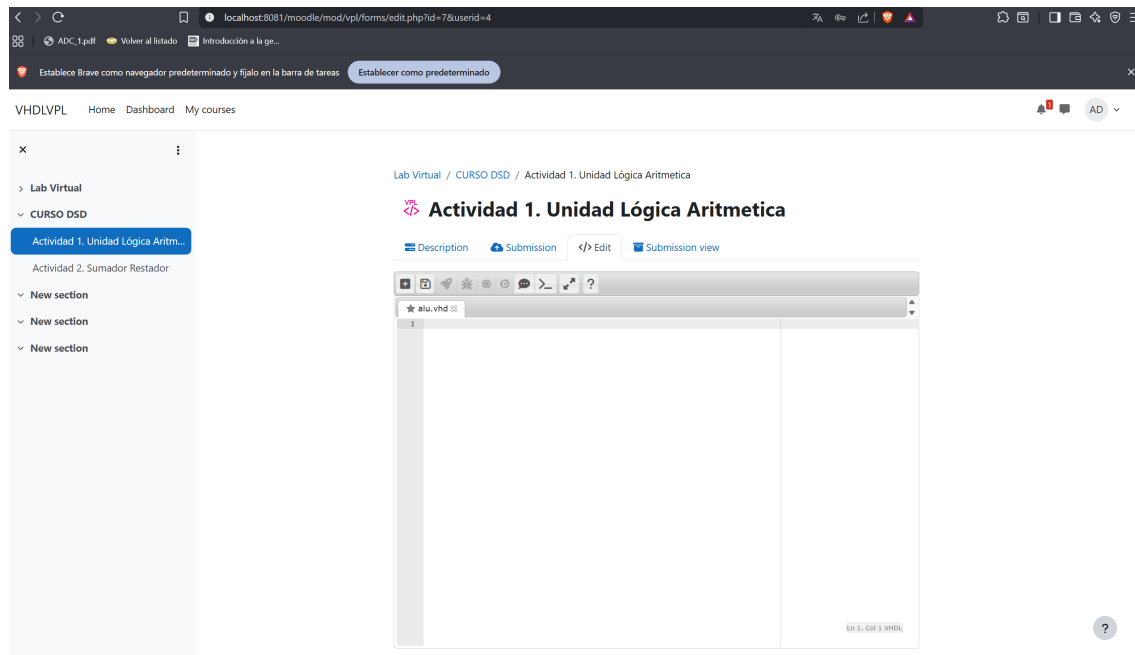


Figura 3.1: Archivo alu.vhd creado y visible en el editor de VPL

Paso 4

Escribir el código VHDL en el área del editor. La Figura 3.2 muestra un ejemplo de diseño de una Unidad Lógico-Aritmética (ALU) con su entidad y arquitectura completas.

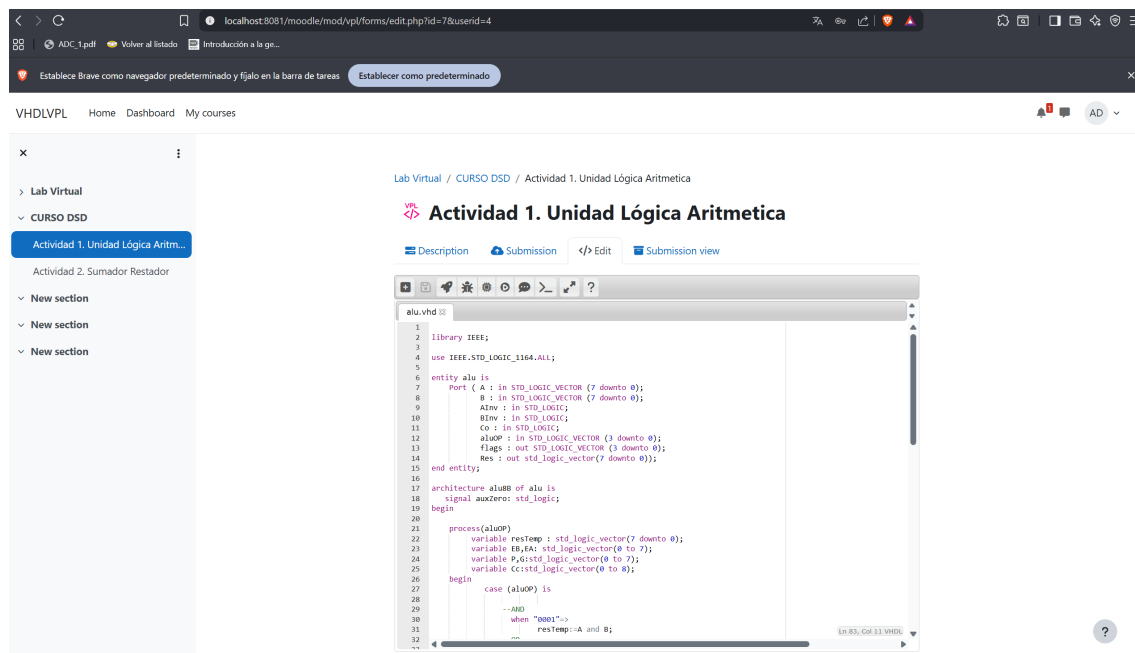


Figura 3.2: Código VHDL escrito en el archivo alu.vhd

3.3 Nomenclatura y organización de archivos

Se recomienda usar nombres descriptivos en minúsculas separando palabras con guion bajo (por ejemplo: `sumador_restador.vhd`, `mux_4to1.vhd`). Evitar espacios y caracteres especiales en los nombres de archivo, ya que pueden causar errores durante la compilación.

Compilar Archivos VHDL

4.1 Proceso de compilación en VPL

La acción **Run** compila el diseño VHDL y muestra el resultado en el panel *Compilation*. Para ejecutarla:

Paso 1

Con el archivo VHDL abierto en el editor, hacer clic en el ícono **Run** (triángulo de reproducción) en la barra de herramientas.

Paso 2

Esperar a que el servidor procese la compilación. El panel de resultados muestra el progreso con mensajes **SUCCESS:=>** para cada etapa completada correctamente (Figura 4.1).

CAPÍTULO 4. COMPILAR ARCHIVOS VHDL

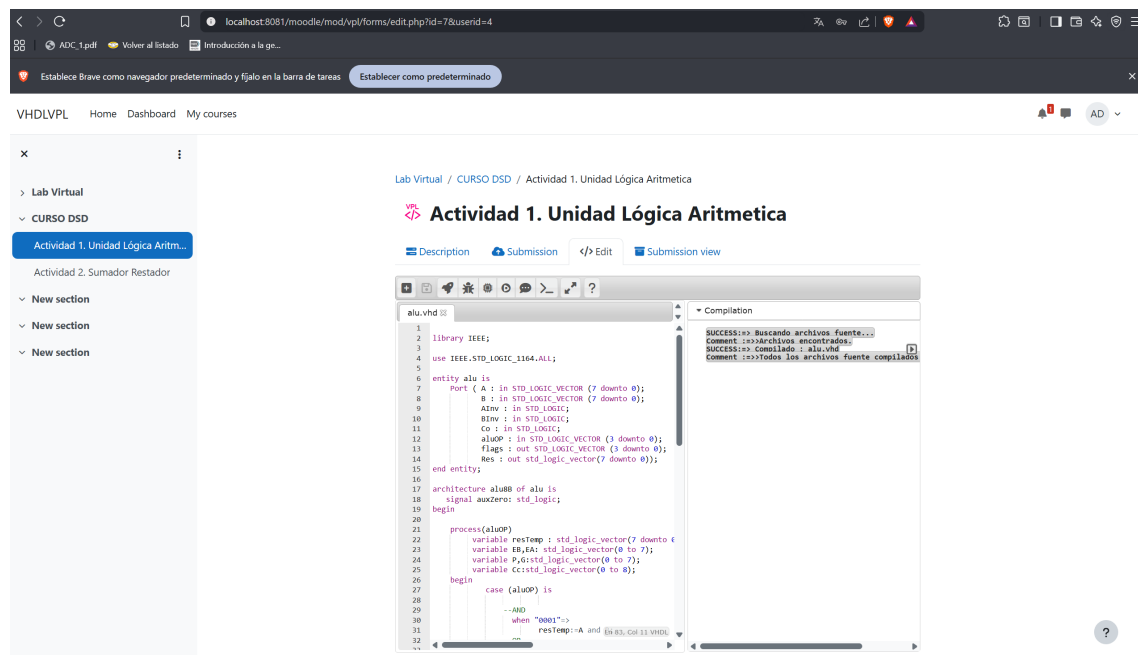


Figura 4.1: Resultado de compilación exitosa en el panel de resultados

Para visualizar el código y los resultados simultáneamente, activar la vista en pantalla completa desde la barra de herramientas (Figura 4.2).

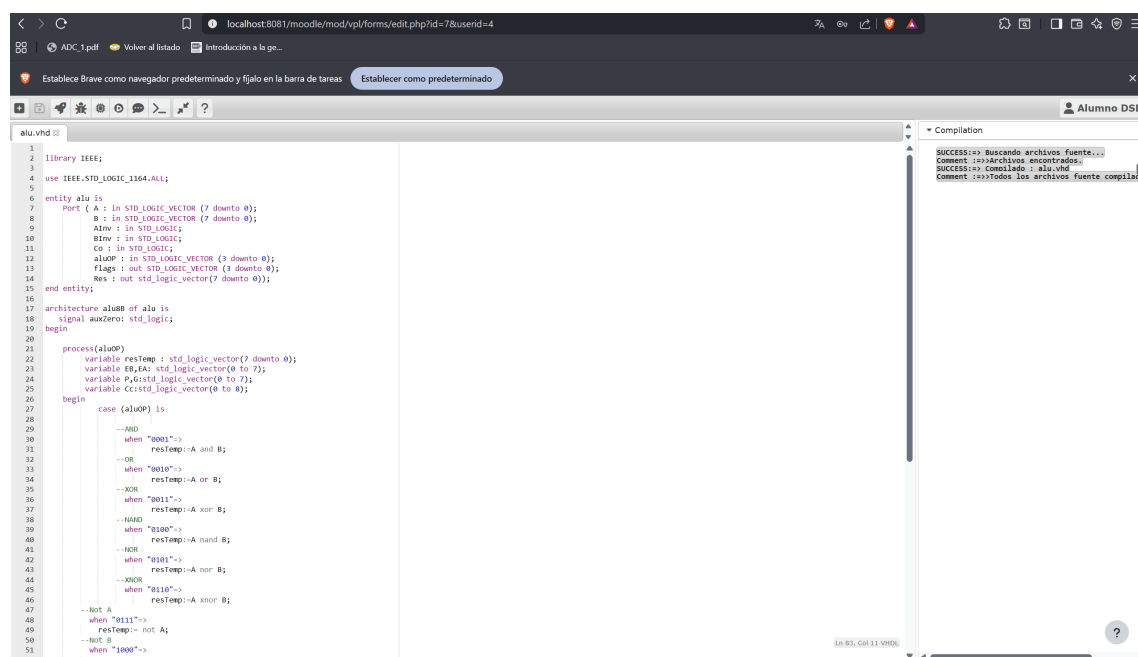


Figura 4.2: Vista en pantalla completa del editor con compilación exitosa

4.2 Interpretación de errores y advertencias

Si el código contiene errores, el panel los reporta con el prefijo **ERROR:=>** seguido de la descripción del problema y la línea donde se encontró (Figura 4.3).

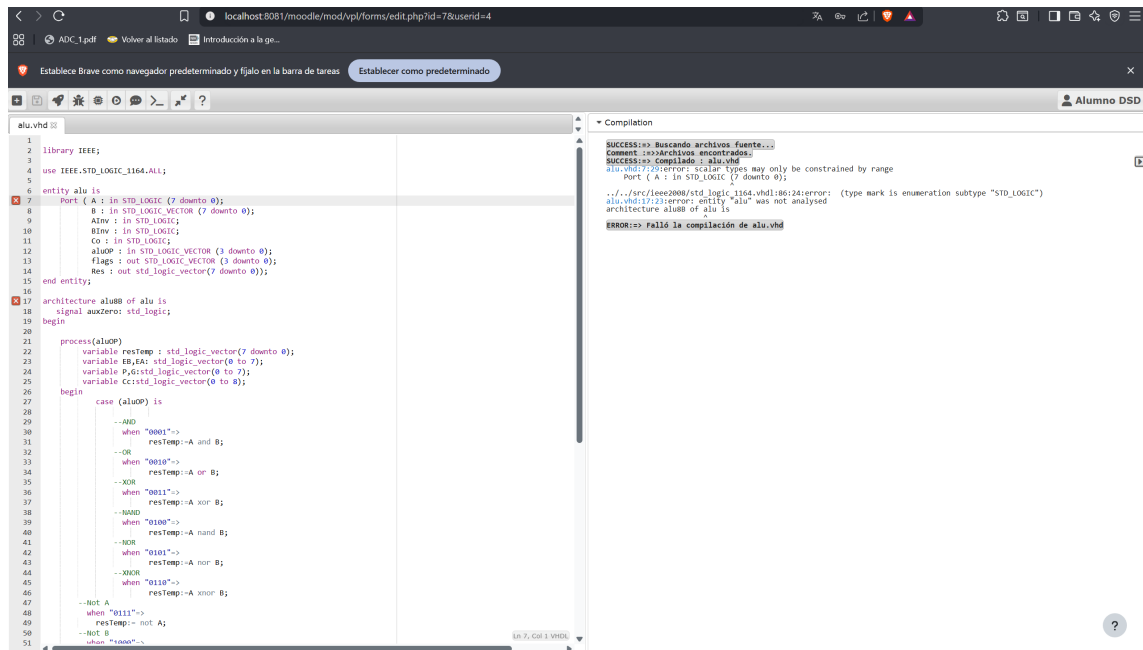


Figura 4.3: Panel de resultados mostrando errores de compilación

Los mensajes de error de GHDL siguen el formato:

archivo.vhd:<línea>:<columna>: error: <descripción>

Para corregir un error, localizar la línea indicada en el mensaje, revisar la sintaxis en esa sección y ejecutar **Run** nuevamente para verificar la corrección.

4.3 Buenas prácticas de compilación

- Compilar frecuentemente mientras se escribe el código, en lugar de esperar a tener el diseño completo.
- Corregir errores de arriba a abajo: un error temprano puede generar errores secundarios en líneas posteriores.
- Verificar que el nombre de la entidad coincida exactamente con el nombre del archivo .vhd.

Generación de Código Testbench

5.1 ¿Qué es un testbench?

Un *testbench* es un archivo VHDL especial cuyo propósito es instanciar el componente bajo prueba (*Unit Under Test*, UUT) y aplicarle estímulos (vectores de prueba) para verificar su comportamiento. El testbench no sintetiza en hardware; solo se utiliza en simulación.

5.2 Generación automática en el entorno VPL

VPL genera automáticamente un testbench básico a partir del archivo VHDL fuente. Para utilizarlo:

Paso 1

Asegurarse de que el archivo VHDL fuente contiene código válido y compila sin errores.

Archivo vacío

Si se intenta generar el testbench con el archivo fuente vacío, VPL muestra el error “*The file ‘archivo.vhd’ is empty. Add VHDL content before generating a testbench.*”, como se observa en la Figura 5.1. Es necesario escribir al menos la entidad y la arquitectura antes de generar el testbench.

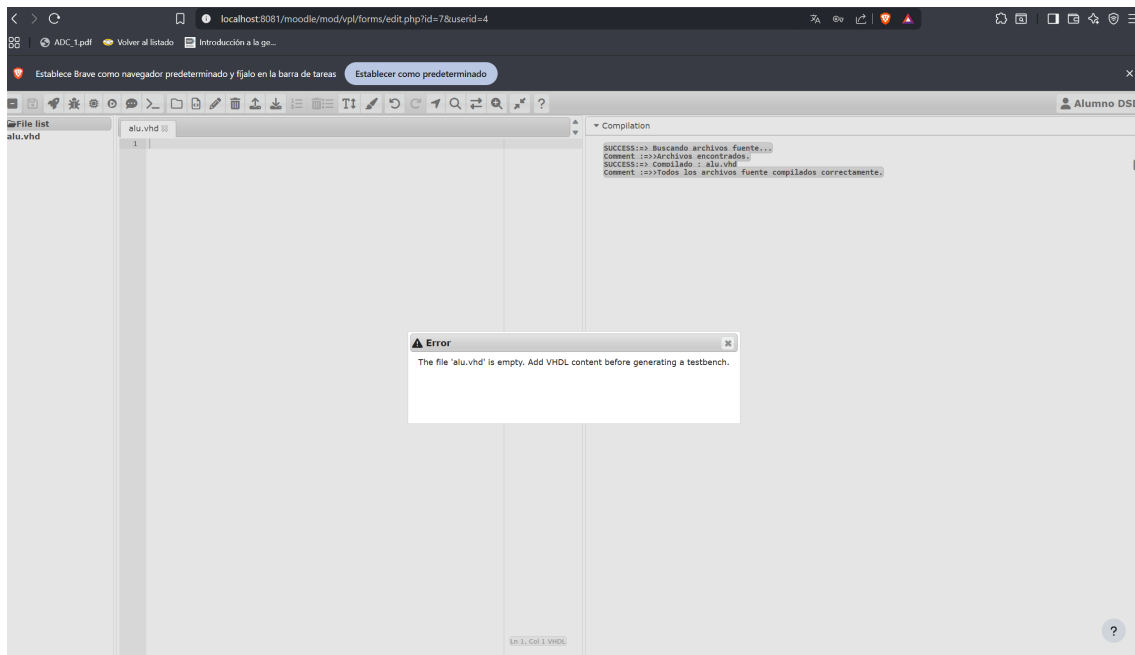


Figura 5.1: Error al intentar generar testbench desde un archivo VHDL vacío

Paso 2

Hacer clic en el ícono **Generate testbench** en la barra de herramientas del editor.

Paso 3

VPL analiza la entidad del archivo fuente, extrae sus puertos y genera automáticamente un archivo `<nombre>_tb.vhd` con la estructura básica: declaración de señales, instanciación del componente (*port map*) y un proceso `stim_proc` con el marcador - TODO: add test vectors here (Figura 5.2).

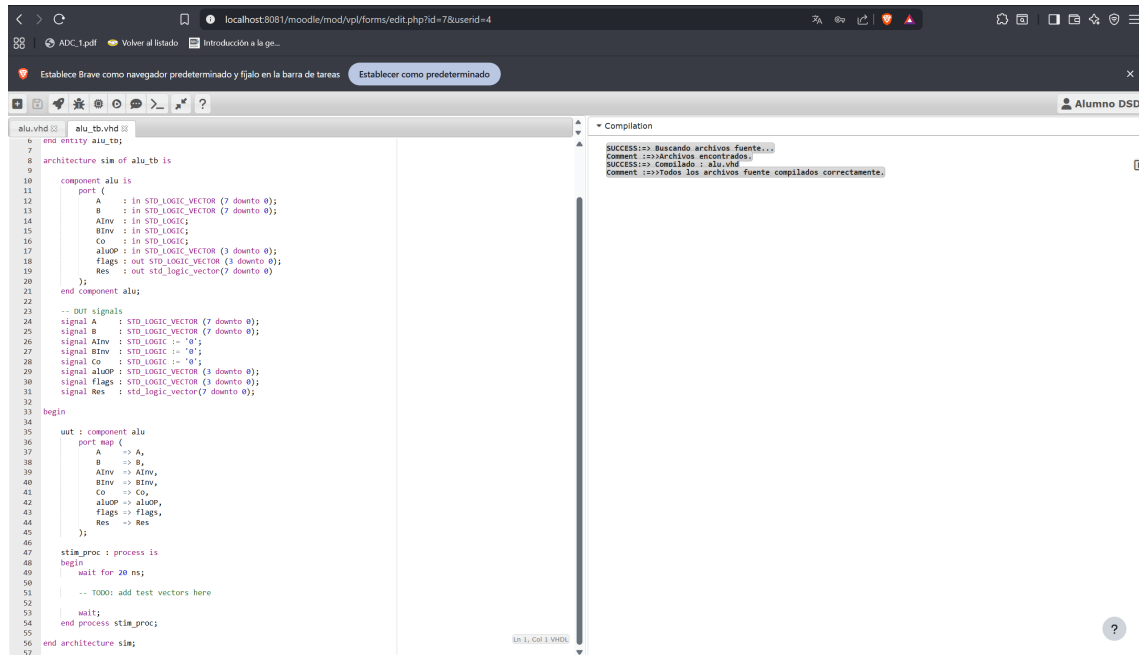


Figura 5.2: Testbench generado automáticamente para el diseño VHDL

5.3 Personalización del testbench generado

El testbench generado incluye la estructura necesaria, pero los vectores de prueba deben añadirse manualmente dentro del proceso `stim_proc`. Cada vector consiste en asignar valores a las señales de entrada y esperar un tiempo antes del siguiente estímulo:

```

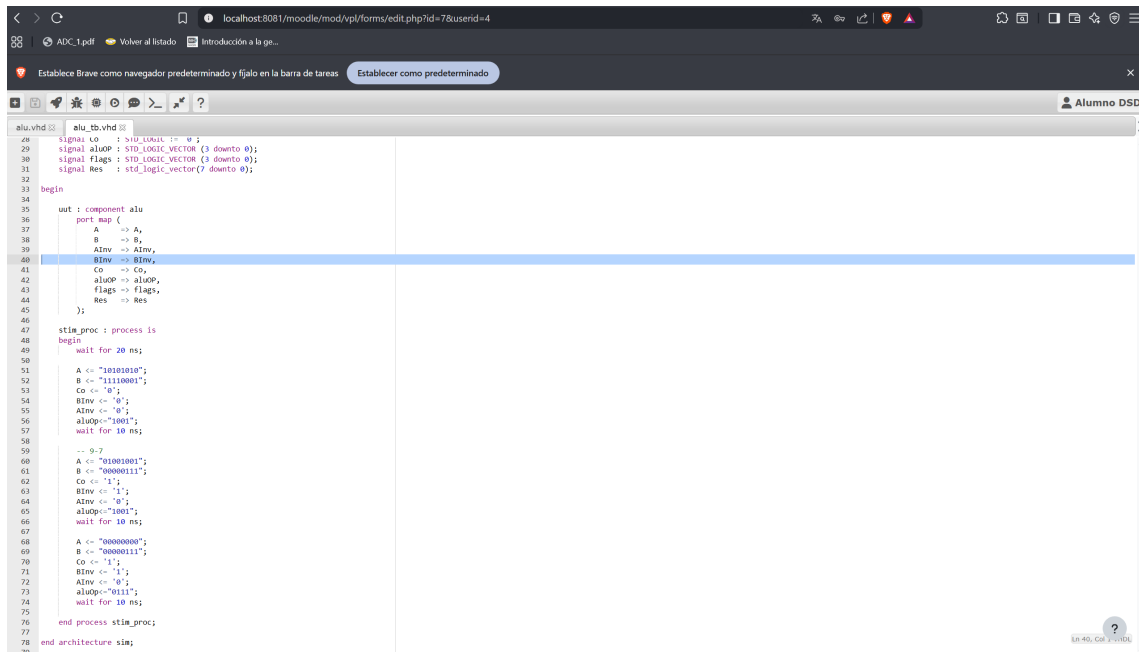
1  stim_proc: process is
2  begin
3      -- Vector 1
4      A    <= "10101010";
5      B    <= "11110001";
6      Co   <= '0';
7      aluP <= "0001";
8      wait for 20 ns;
9
10     -- Vector 2
11     A    <= "00001111";
12     B    <= "00000001";
13     Co   <= '1';
14     aluP <= "0001";
15     wait for 20 ns;
16
17     wait;
18 end process stim_proc;

```

Listing 5.1: Ejemplo de vectores de prueba en el testbench

CAPÍTULO 5. GENERACIÓN DE CÓDIGO TESTBENCH

La Figura 5.3 muestra el testbench con varios vectores de prueba ya añadidos en el proceso `stim_proc`.



```
alu.vhd | alu_tb.vhd |
28 signal Co : STD_LOGIC := '0';
29 signal aluOP : STD_LOGIC_VECTOR (3 downto 0);
30 signal Flags : STD_LOGIC_VECTOR (3 downto 0);
31 signal Res : std_logic_vector(7 downto 0);
32
33 begin
34
35   uut : component alu
36     port map (
37     A => A,
38     B => B,
39     Altnv => Altnv,
40     BInv => BInv,
41     Co => Co,
42     aluOP => aluOP,
43     flags => flags,
44     Res => Res
45   );
46
47   stim_proc : process is
48   begin
49     wait for 20 ns;
50
51     A <= "10101010";
52     B <= "11110001";
53     Co <= '0';
54     BInv <= '0';
55     Altnv <= '0';
56     aluOP <= "1001";
57     wait for 10 ns;
58
59     -- 9-7
60     A <= "01001001";
61     B <= "00000111";
62     Co <= '1';
63     BInv <= '1';
64     Altnv <= '0';
65     aluOP <= "1001";
66     wait for 10 ns;
67
68     A <= "00000000";
69     B <= "00000111";
70     Co <= '1';
71     BInv <= '1';
72     Altnv <= '0';
73     aluOP <= "0111";
74     wait for 10 ns;
75
76   end process stim_proc;
77
78 end architecture sim;
79
```

Figura 5.3: Testbench con vectores de prueba añadidos manualmente

Uso de la Simulación en VPL

6.1 Iniciar la simulación

La acción **Debug** compila el diseño junto con su testbench y abre la herramienta **GTKWave** para visualizar las formas de onda. Antes de iniciarla, verificar que:

- El archivo fuente VHDL compile sin errores.
- El testbench `_tb.vhd` esté creado y contenga vectores de prueba con sentencias `wait`.

Paso 1

Hacer clic en el ícono **Debug** en la barra de herramientas del editor.

Paso 2

VPL compila ambos archivos, genera el archivo de simulación (`.ghw`) y lanza GTKWave. El panel *Compilation* muestra el progreso detallado con mensajes `SUCCESS:=>` e `INFO:=>` sobre la clasificación de archivos, detección de la entidad principal, tiempo de simulación y proyecto GTKWave generado (Figura 6.1).

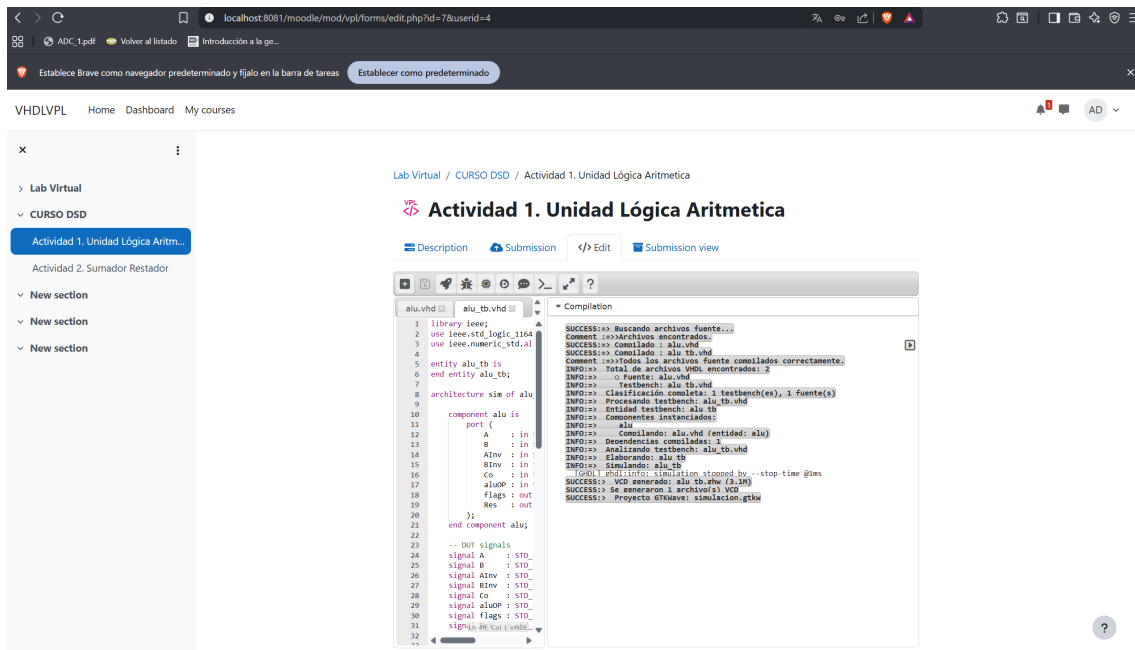


Figura 6.1: Salida detallada del proceso de compilación al ejecutar Debug

Paso 3

Una vez completado el proceso, GTKWave se abre mostrando el árbol de señales (*SST*) a la izquierda y el área de formas de onda a la derecha (Figura 6.2). El área de formas de onda aparece inicialmente vacía hasta que se añadan señales.

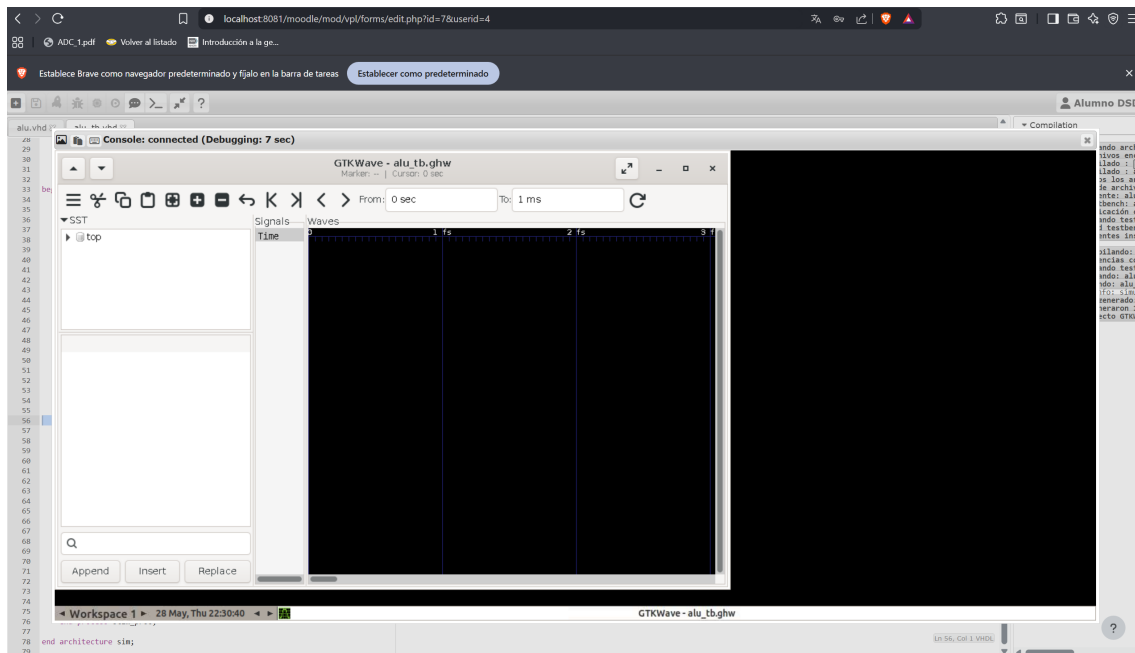


Figura 6.2: GTKWave abierto con la jerarquía de señales y área de visualización vacía

6.2 Visualización de formas de onda

Paso 1

En el panel SST (izquierda), expandir la jerarquía haciendo clic sobre los nodos hasta llegar a las señales del circuito bajo prueba (Figura 6.3).

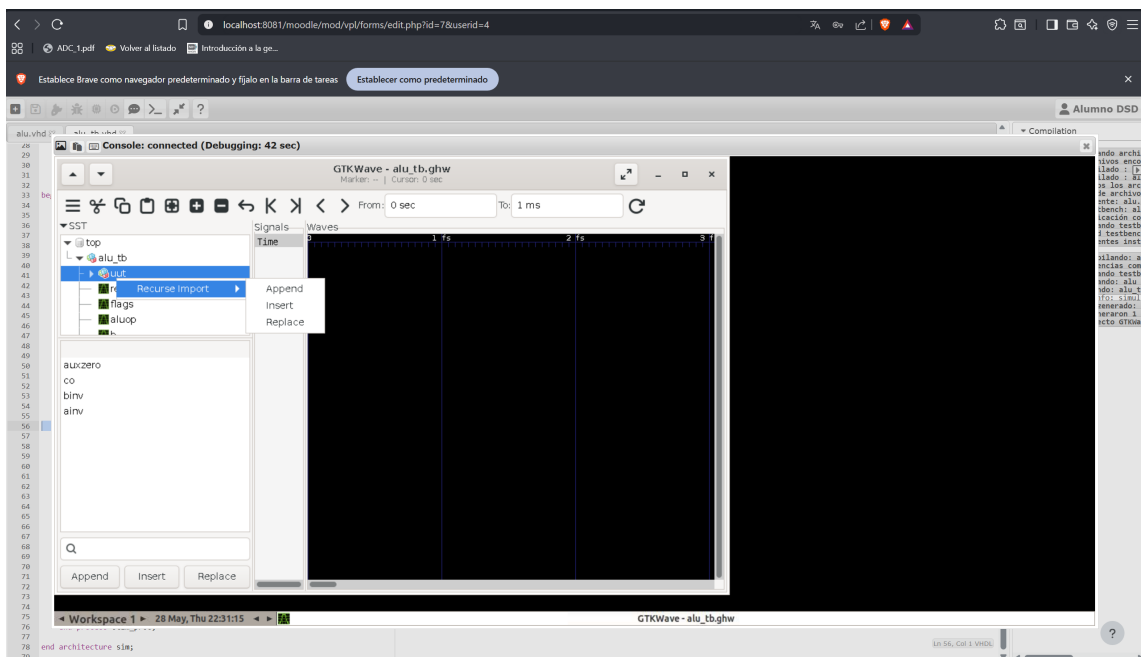


Figura 6.3: Árbol de señales SST expandido con menú contextual para importar señales

Paso 2

Hacer clic derecho sobre el componente deseado y seleccionar **Recurse Import** → **Append** para añadir todas sus señales al área de visualización.

Paso 3

Si GTKWave muestra el cuadro de diálogo “*Really import N facilities?*” (Figura 6.4), hacer clic en **Yes** para confirmar la importación.

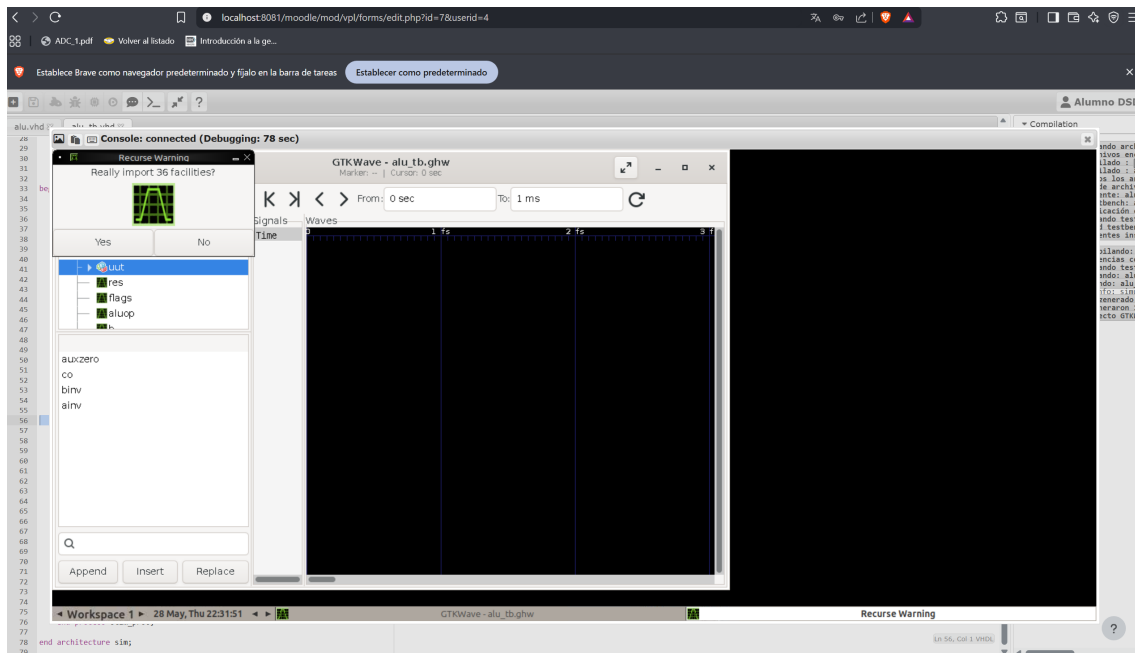


Figura 6.4: Diálogo de confirmación para importar todas las señales del componente

Paso 4

Las señales aparecen en el panel izquierdo con sus formas de onda en el área de visualización (Figura 6.5).

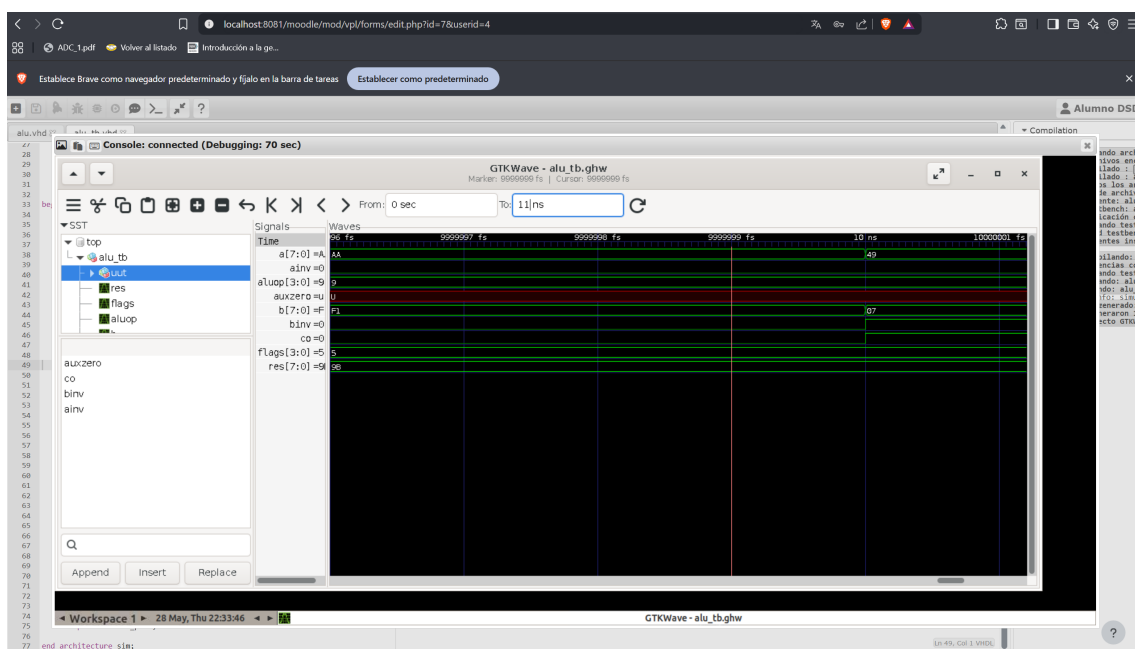


Figura 6.5: Formas de onda de las señales del diseño visualizadas en GTKWave

Las señales de tipo vector (por ejemplo `a[7:0]`, `res[7:0]`) se muestran por defecto como bus. Para visualizar cada bit individualmente, hacer clic en la flecha de

expansión junto al nombre de la señal (Figura 6.6).

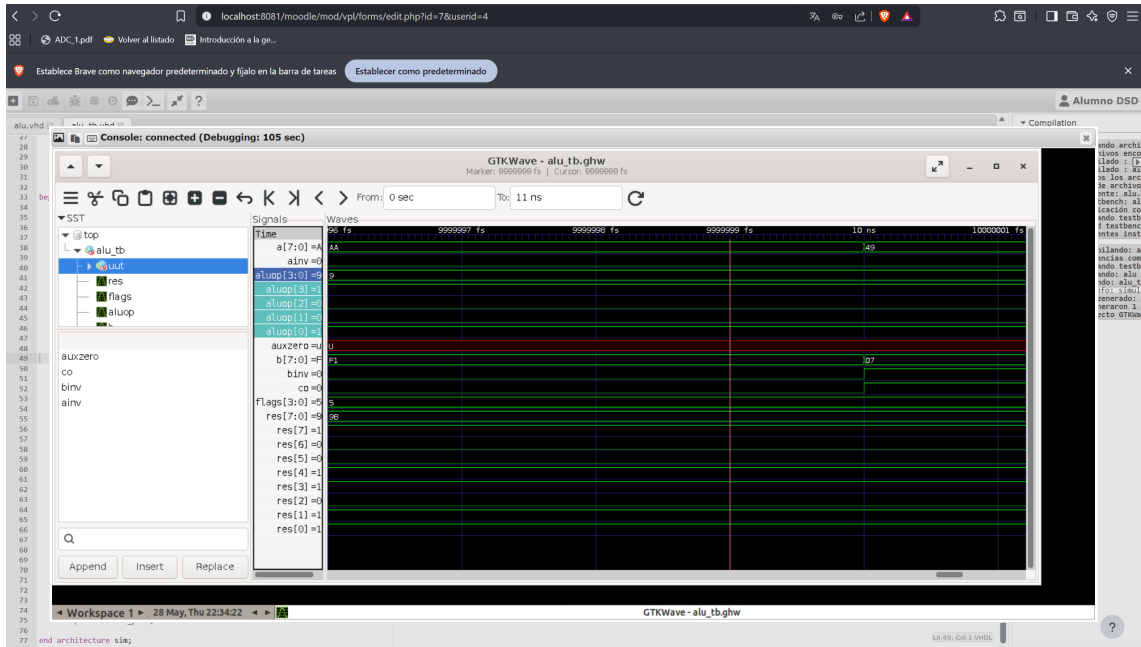


Figura 6.6: Señales vectoriales expandidas mostrando cada bit individualmente en GTKWave

6.3 Depuración mediante simulación

GTKWave permite inspeccionar el valor de cada señal en cualquier instante haciendo clic sobre el área de formas de onda. El valor aparece en la columna junto al nombre de la señal. Para navegar entre eventos, usar los controles de avance y retroceso de la barra de herramientas de GTKWave.

Para controlar el tiempo total de simulación, añadir la directiva `vpl_stop_time` como comentario en las primeras líneas del testbench `_tb.vhd`:

```
-- vpl_stop_time = 2 ms
```

Listing 6.1: Directiva para establecer el tiempo de simulación

La Figura 6.7 muestra el testbench con esta directiva y la salida del compilador confirmando que el tiempo fue detectado correctamente.

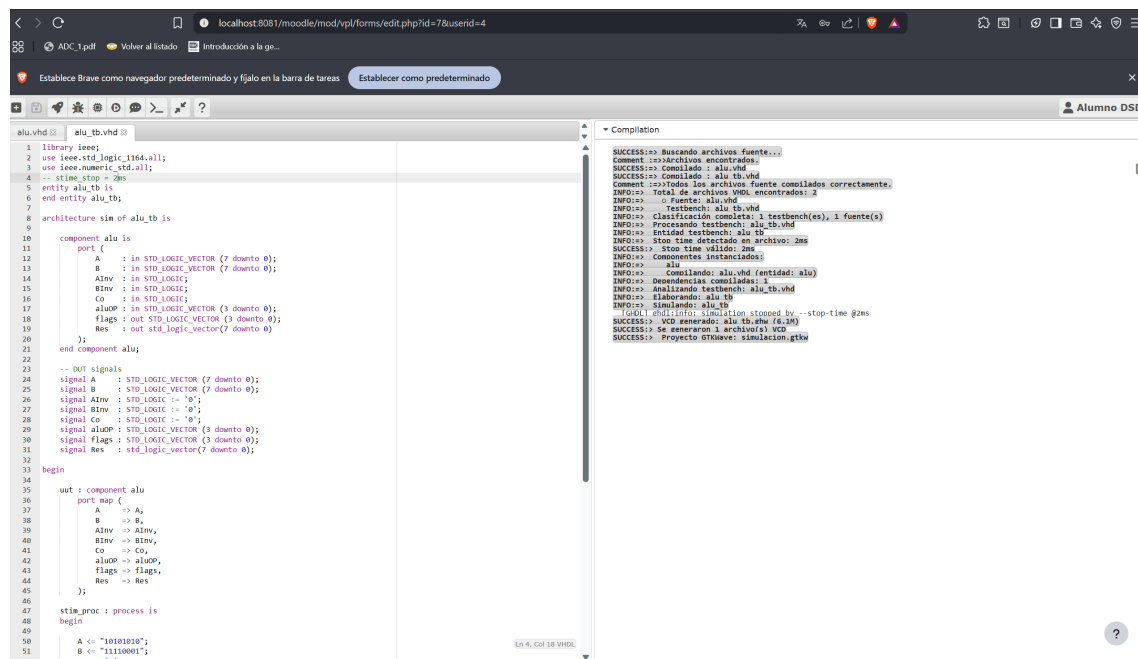


Figura 6.7: Directiva `vpl_stop_time` declarada en el testbench y confirmada por el compilador

Al lanzar la simulación con esta directiva, GTKWave ajusta el eje temporal hasta el valor especificado (Figura 6.8), lo que permite visualizar con mayor resolución el comportamiento del circuito durante el intervalo de interés.

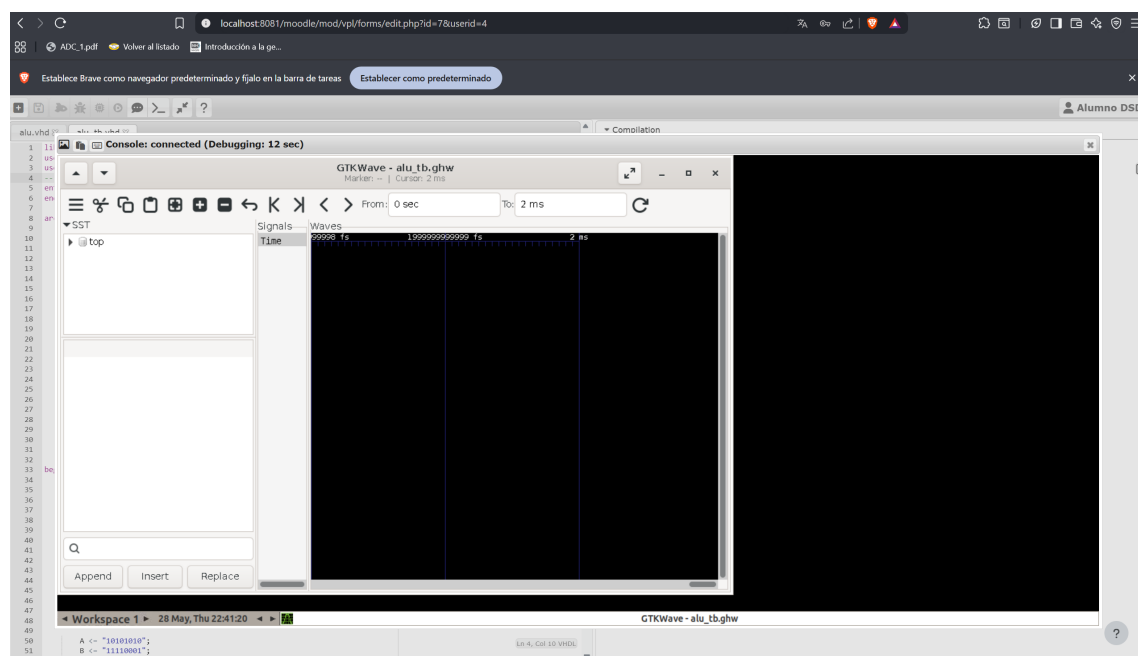


Figura 6.8: GTKWave con el eje temporal ajustado al tiempo definido por `vpl_stop_time`

Uso de la Evaluación Automática

7.1 ¿Cómo funciona la evaluación automática?

Opcional

Esta opción por default esta deshabilitada para el alumno, el profesor puede habilitarla o no dependiendo de la configuración de la actividad. Si la evaluación automática no está habilitada, el alumno tendra que esperar a que el profesor evalúe su actividad para poder ver los resultados y la calificación obtenida.

Al enviar el diseño para evaluación, VPL ejecuta el archivo VHDL del alumno contra los casos de prueba definidos por el profesor. Para cada caso, el evaluador:

1. Compila el diseño del alumno con GHDL.
2. Genera un testbench automático a partir de los casos de prueba.
3. Simula el diseño y compara las salidas obtenidas con las esperadas.
4. Calcula la calificación final aplicando las penalizaciones definidas para cada caso fallido.

El resultado aparece en el panel de comentarios con el detalle de qué casos pasaron (PASS) y cuáles fallaron (FAIL), junto con la calificación final.

7.2 Enviar el diseño para evaluación

Paso 1

Verificar que el diseño compile sin errores utilizando la acción **Run** antes de enviarlo.

Paso 2

Hacer clic en el ícono **Evaluate** en la barra de herramientas. VPL procesará el diseño y mostrará la calificación obtenida en el panel de resultados.

Paso 3

Una vez evaluado, el diseño queda registrado como entrega en *Moodle*. La Figura 7.1 muestra la vista de entrega con la fecha y hora de envío y el enlace para descargar el archivo entregado.

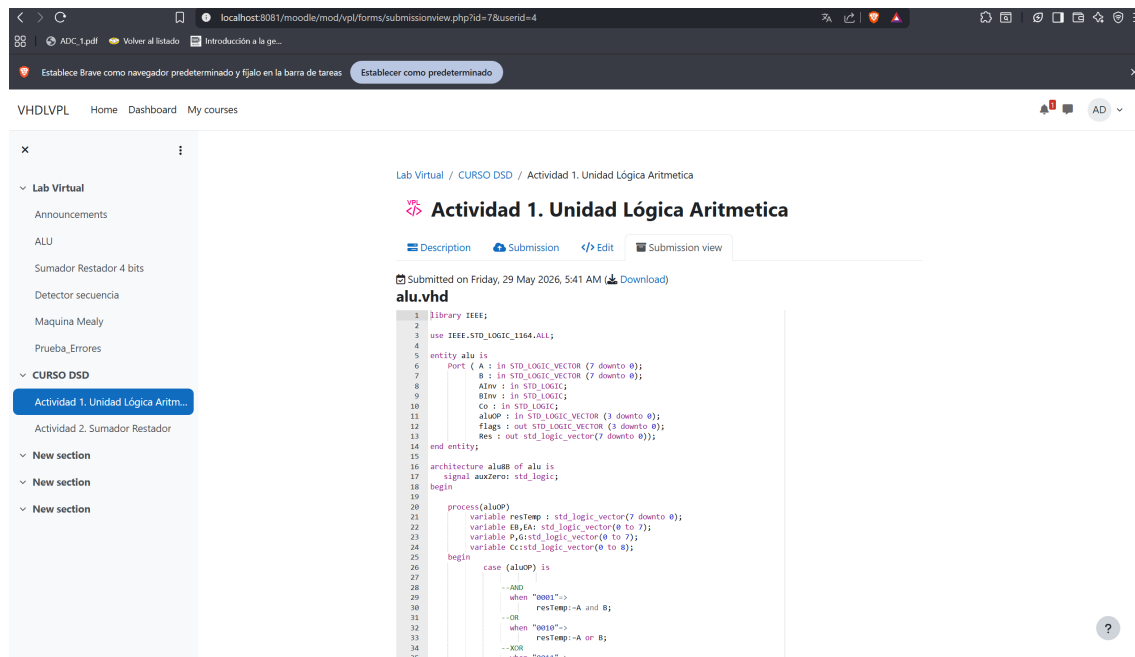


Figura 7.1: Vista de entrega en Moodle con fecha de envío y archivo registrado

7.3 Interpretar la retroalimentación

El reporte de evaluación indica:

- **PASS:** el caso de prueba fue superado correctamente.
- **FAIL:** la salida obtenida no coincide con la esperada en ese caso.
- **Nota:** calificación final calculada sobre 100 puntos.
- **Evaluation output:** resumen con el número de casos totales, correctos y fallidos.

Leer los mensajes de **FAIL** para identificar qué señales presentaron valores incorrectos y en qué momento de la simulación ocurrieron, lo que orienta la corrección del diseño.

7.4 Reintentos y límite de envíos

El número máximo de entregas está determinado por la configuración del profesor. Se recomienda:

- Usar **Run** para verificar la compilación antes de cada envío.
- Depurar el comportamiento del diseño con **Debug** antes de gastar un intento de evaluación.
- Revisar los mensajes de retroalimentación de intentos anteriores para corregir los casos que fallaron.

Límite de intentos

Si la actividad tiene un número limitado de evaluaciones, cada clic en **Evaluate** consume un intento sin importar el resultado obtenido. Asegúrese de haber depurado el diseño antes de evaluar.

Parte II

Guía del Profesor

Creación de Actividad VPL

8.1 Precondiciones

Antes de crear una actividad VPL, verificar que se cumplan los siguientes requisitos:

- **Cuenta con rol de profesor y permisos de edición:** el usuario debe tener asignado el rol de *Profesor* en *Moodle* con capacidad de editar el curso. Sin este rol, las opciones de configuración de actividades no estarán disponibles.
- **Sesión activa:** el profesor debe haber iniciado sesión en la plataforma *Moodle* antes de realizar cualquier operación.
- **Curso creado:** debe existir al menos un curso en *Moodle* al cual el profesor tenga acceso de edición. La actividad VPL se agrega dentro de un curso existente.
- **Plugin de VPL instalado:** El plugin de VPL debe estar instalado y activo en la plataforma *Moodle*.

Verificación de permisos

Si las opciones de edición no aparecen en el curso, contactar al administrador de la plataforma para revisar el rol asignado a la cuenta.

8.2 Crear una actividad VPL

Para crear una actividad VPL en un curso de *Moodle*, seguir estos pasos:

Paso 1

Ingresa al curso donde se desea agregar la actividad VPL.

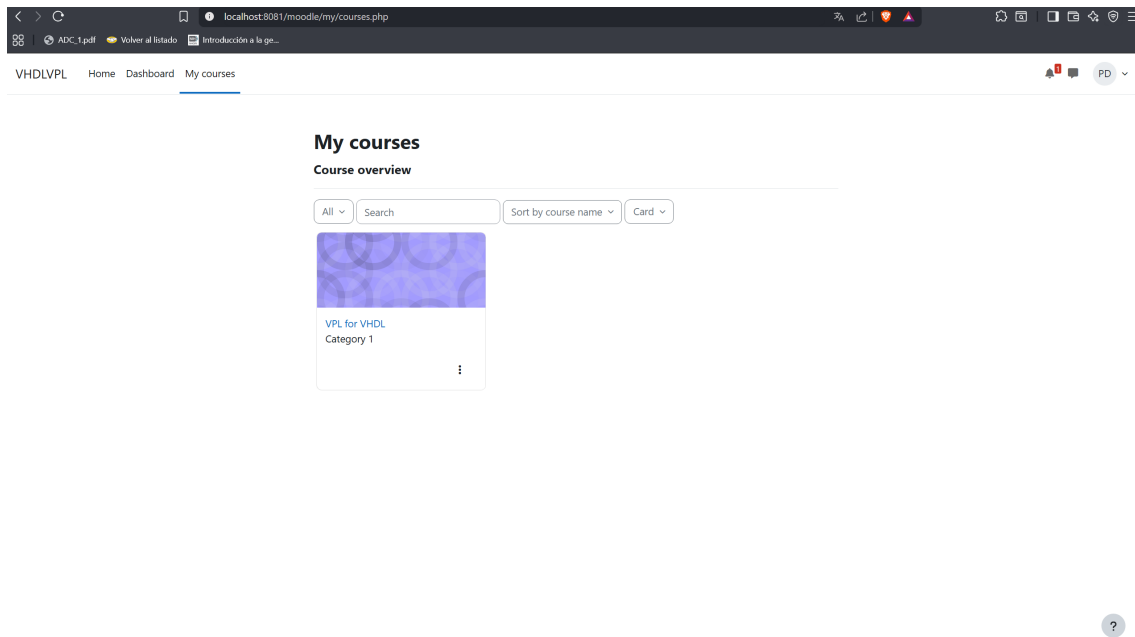


Figura 8.1: Cursos disponibles en Moodle

Paso 2

Activar el modo de edición del curso haciendo clic en el botón **Activar edición**.

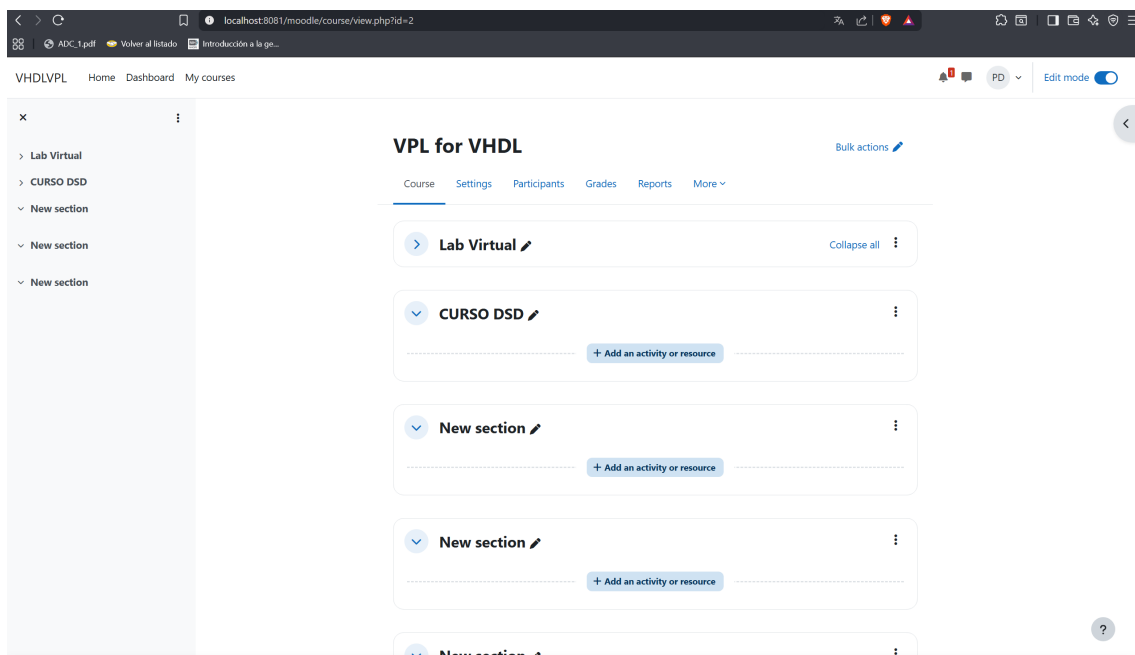


Figura 8.2: Cursos donde se desea agregar la actividad VPL

Paso 3

En la sección del curso donde se desea colocar la actividad, hacer clic en **Añadir una actividad o recurso**. En la ventana emergente, seleccionar **Virtual Programming Lab (VPL)**.

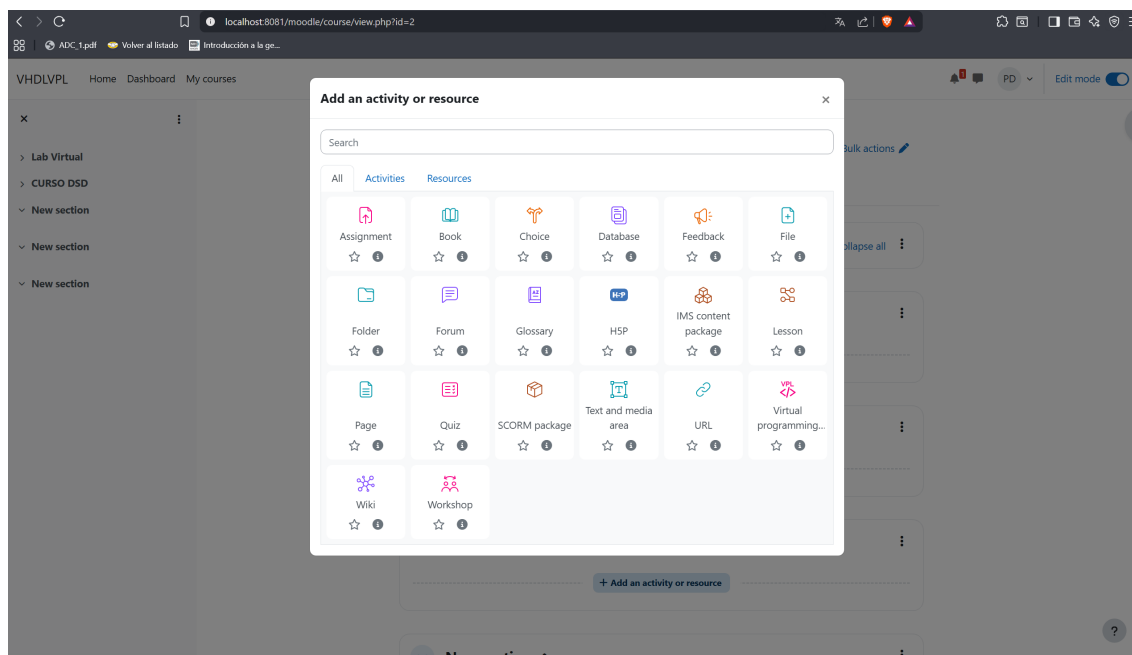


Figura 8.3: Crear actividad VPL

Una vez creada la actividad VPL, se nos muestra una vista para la configuración principal de la actividad, donde se pueden establecer diferentes configuraciones, de las cuales mencionaremos las mas relevantes para el correcto funcionamiento de la actividad, sin embargo, se recomienda revisar cada una de las opciones para personalizar la actividad a las necesidades del curso.

8.2.1 Configuraciones generales

Incluye el nombre de la actividad (obligatorio), una descripción corta y una descripción detallada donde podemos añadir mas recursos (texto, imagenes, videos, etc.).

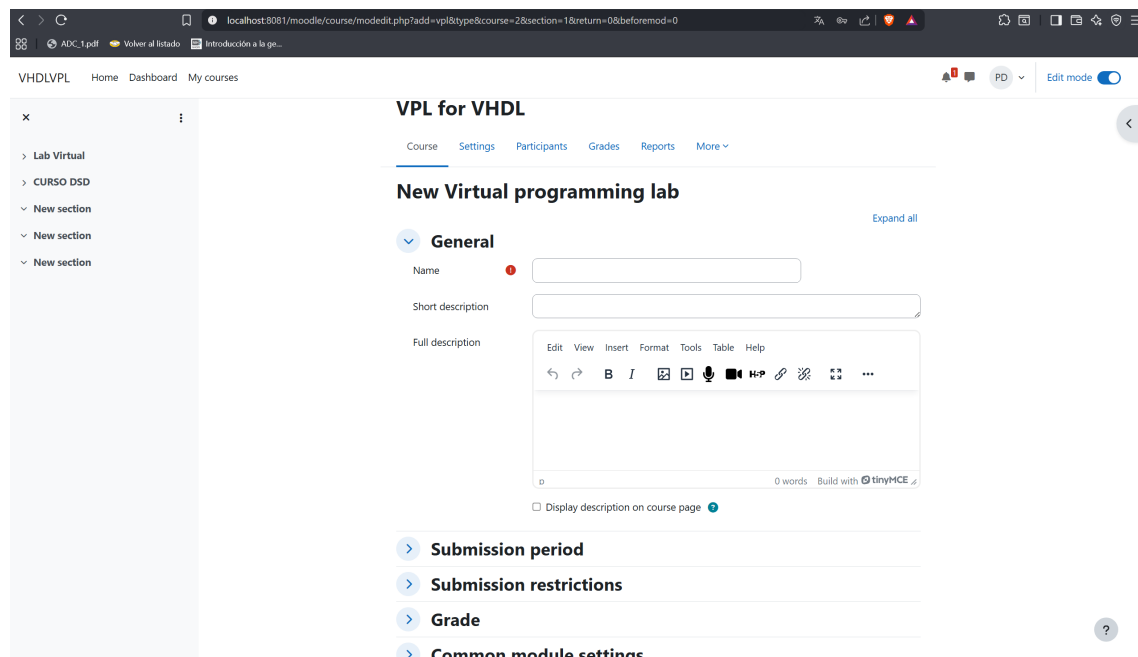


Figura 8.4: Configuraciones generales de la actividad VPL

8.2.2 Restricciones de entrega

En esta sección se pueden establecer configuraciones como el número máximo de archivos que el alumno puede entregar, el tipo de trabajo (individual o grupal), fecha de entrega, entre otras opciones que pueden ser útiles para controlar la entrega de los trabajos por parte de los alumnos. Aquí lo más importante es configurar el número máximo de archivos que el alumno puede entregar, por default se establecieron 2 archivos máximos, considerando un archivo vhdl fuente y su archivo testbench generado, sin embargo, dependiendo de la actividad se pueden establecer más archivos máximos para que el alumno pueda entregar archivos adicionales (por ejemplo, un archivo de texto con una explicación del diseño o un diagrama del circuito).

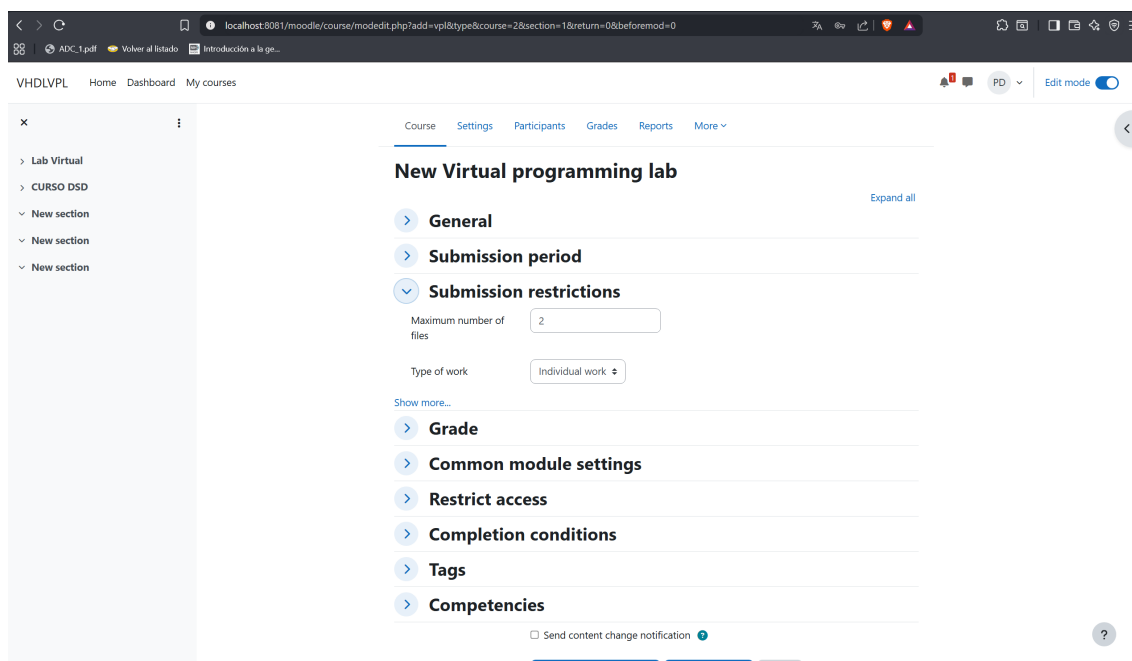


Figura 8.5: Restricciones de entrega en la actividad VPL

8.2.3 Otras configuraciones

EL VPL es un plugin muy completo que ofrece muchas opciones de configuración, por lo que se recomienda revisar cada una de las secciones para personalizar la actividad a las necesidades del curso, sin embargo, se recomienda revisar la documentación propia de VPL for Moodle para saber más acerca de las otras configuraciones.

Una vez configurada la actividad, se debe guardar los cambios para que la actividad quede disponible para los alumnos.

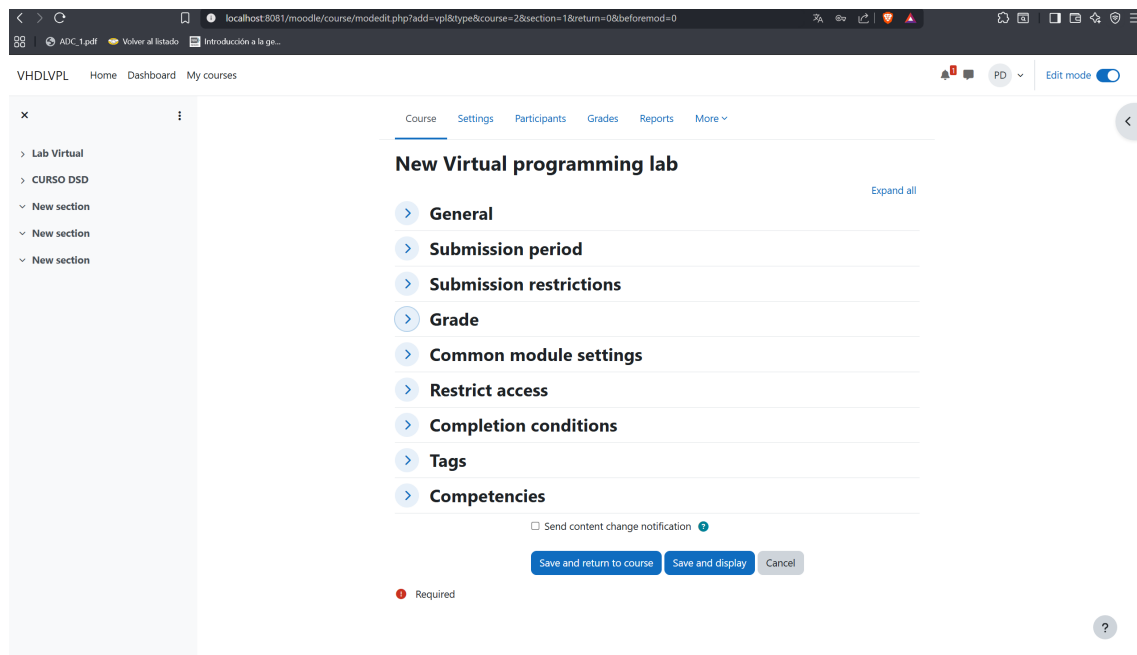


Figura 8.6: Guardar cambios en la actividad VPL

Una vez guardadas las configuraciones establecidas, la actividad debería verse como en la siguiente imagen:

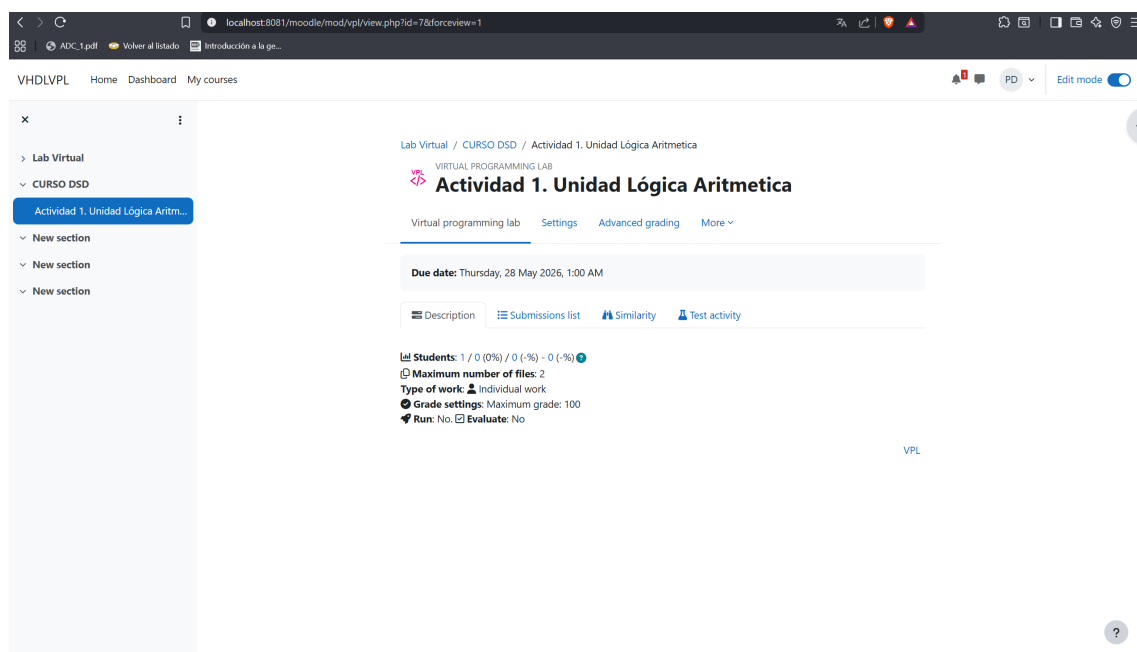


Figura 8.7: Visión general de la actividad VPL

Acciones Permitidas

En esta sección se describen como configurar las acciones que ofrece el entorno de VPL, esto respecto a la ejecución, depuración y evaluación de los diseños VHDL entregados por los alumnos. Es importante configurar esta sección una vez creada la actividad, ya que si visualizamos la imagen 8.7, observamos que por se visualiza la acción RUN habilitada y Evaluate deshabilitada, por lo que hay que configurar las acciones para permitir o no al alumno usarlas.

9.1 Tipos de acciones en VPL

En el laboratorio virtual de VPL, existen 5 tipos principales de acciones que se pueden configurar para los alumnos:

- **Run (Ejecutar):** Permite a los alumnos ejecutar su código VHDL para verificar su funcionamiento básico. Esta acción generalmente compila y simula el diseño, mostrando los resultados de la simulación.
- **Debug (Depurar):** Permite a los alumnos depurar su código VHDL, lo que puede incluir la visualización de formas de onda, inspección de señales y análisis detallado del comportamiento del diseño durante la simulación.
- **Evaluate (Evaluar):** Permite a los alumnos enviar su diseño para evaluación automática, donde se ejecutan casos de prueba predefinidos para verificar la corrección del diseño y asignar una calificación basada en los resultados.
- **Remote Lab (Laboratorio Remoto):** Permite conectarse al laboratorio remoto de FPGA'S.
- **Testbench (Generar Testbench):** Permite a los alumnos generar automáticamente un testbench básico para su diseño VHDL, lo que facilita la simulación y verificación de su código.

9.2 Configurar acciones disponibles para el alumno

Para habilitar o deshabilitar estas acciones para los alumnos, se deben seguir los siguientes pasos:

Paso 1

Ingresa a la actividad VPL creada y hacer clic en el botón **MAS / More** para que se despliegan mas opciones del VPL.

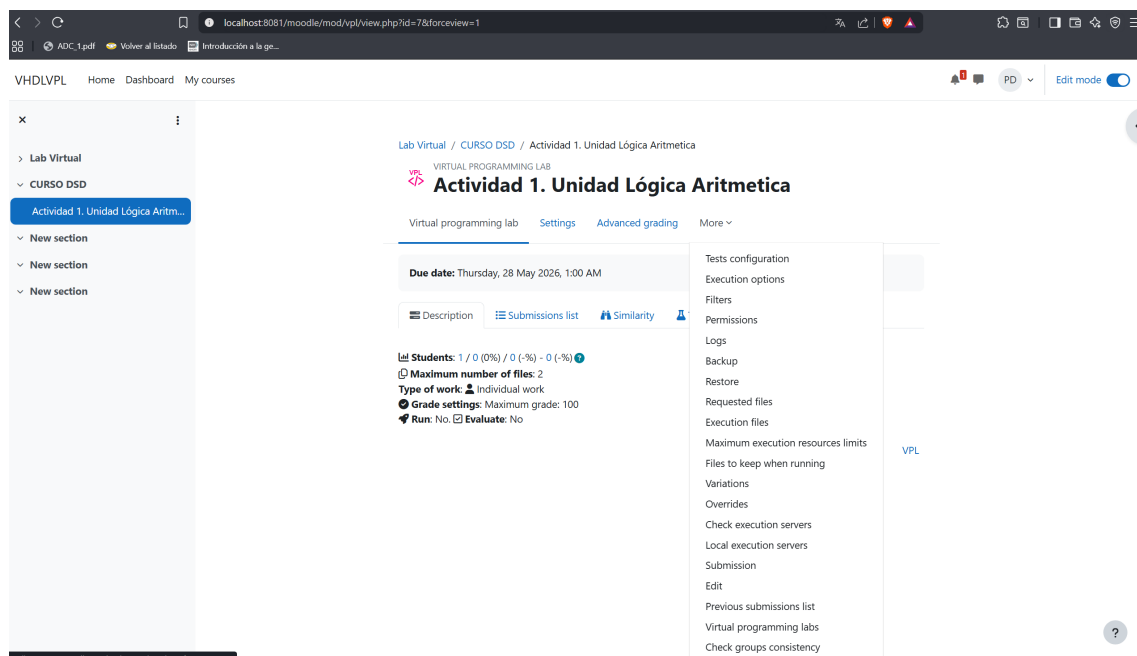


Figura 9.1: Mas configuraciones de la actividad VPL

Paso 2

En la sección de **Acciones de ejecución**, seleccionar las acciones que se desean habilitar para los alumnos.

9.2. CONFIGURAR ACCIONES DISPONIBLES PARA EL ALUMNO

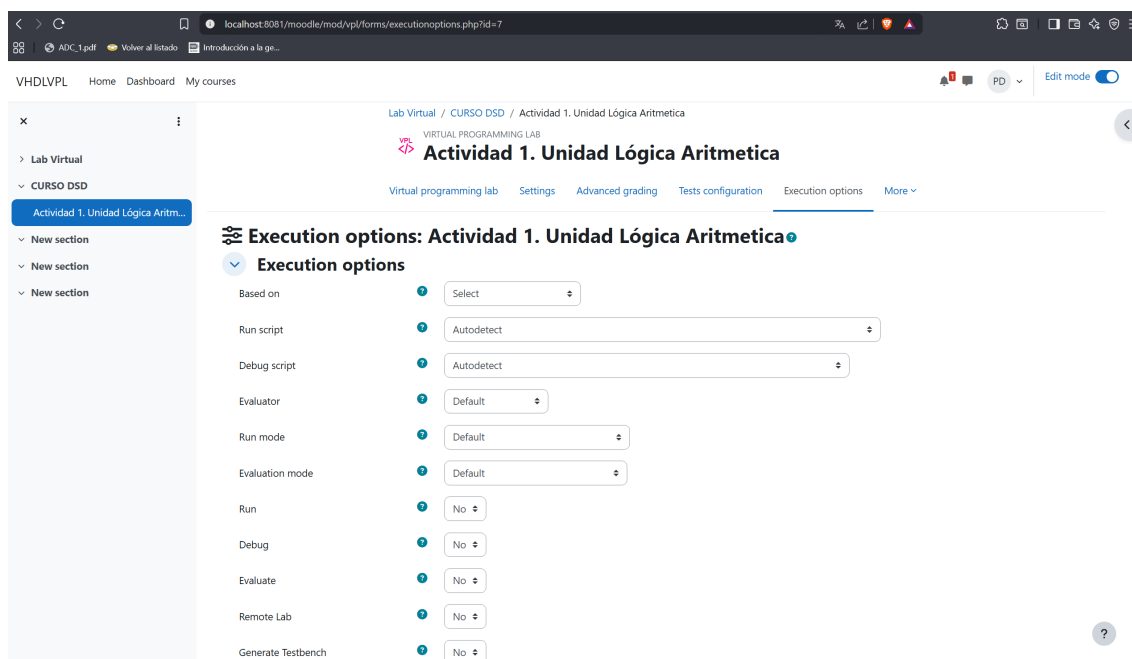


Figura 9.2: Opciones de ejecución en la actividad VPL

Paso 3

Dentro de esta sección lo mas importante es habilitar o deshabilitar las opciones de **Run** , **Debug** , **Remote Lab** , **Generate Testbench** y **Evaluate**, ya que estas son las acciones principales para que el alumno pueda ejecutar.

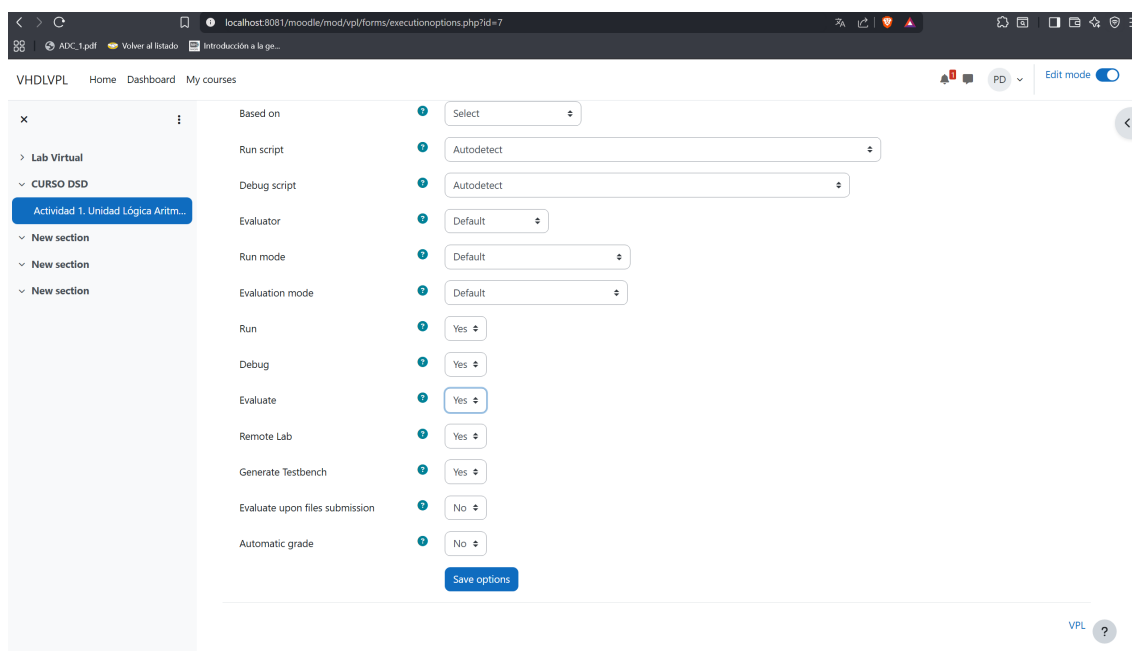


Figura 9.3: Permitir opciones de ejecución en la actividad VPL

Configuración de Casos de Prueba

El archivo `vpl_evaluate.cases` contiene los casos de prueba que el evaluador `vpl_evaluate.sh` ejecuta al calificar un diseño VHDL. El evaluador compila el código del alumno, detecta la entidad top, extrae sus puertos, genera un testbench VHDL automáticamente a partir de los casos, lo simula con GHDL y calcula la nota final.

10.1 Archivo `vpl_evaluate.cases`

El archivo tiene dos bloques diferenciados:

1. **Configuración global** — parámetros que aplican a todos los casos (reloj, reset, tiempo de simulación). Aparecen antes del primer `case`.
2. **Definición de casos** — uno o más bloques `case`, cada uno con sus secciones `input:` y `output:`.

Las líneas que comienzan con `#` son comentarios y se ignoran. Los nombres de parámetro son insensibles a mayúsculas y minúsculas.

10.1.1 Sintaxis general de los casos de prueba

Configuración global

Los parámetros globales se escriben como `parámetro = valor` y se colocan al inicio del archivo, antes de cualquier `case`. La Tabla 10.1 lista los parámetros disponibles.

Cuadro 10.1: Parámetros de configuración global

Parámetro	Ejemplo	Descripción
<code>stop_time_all</code>	<code>1 ms</code>	Tiempo máximo de simulación global.

Parámetro	Ejemplo	Descripción
clock	CLK	Nombre del puerto de reloj. Si se omite, el evaluador lo detecta automáticamente buscando puertos llamados <code>clk</code> , <code>clock</code> o <code>ck</code> .
clock_period	10 ns	Período completo del reloj.
reset	RESET	Nombre del puerto de reset. Si se omite se busca entre <code>reset</code> , <code>rst</code> , <code>clr</code> y variantes con sufijo <code>_n</code> .
reset_active	1	Nivel activo del reset: 1 = activo alto, 0 = activo bajo.
reset_type	async	Tipo de reset: <code>async</code> o <code>sync</code> .
reset_cycles	2	Ciclos de reloj durante los que se mantiene el reset al inicio de la simulación.

Definición de un caso

Cada caso comienza con la línea `case = descripción` y contiene los parámetros opcionales listados en la Tabla 10.2, seguidos de las secciones de señales.

Cuadro 10.2: Parámetros por caso de prueba

Parámetro	Ejemplo	Descripción
case	Prueba AND	Nombre del caso; aparece en los reportes de fallo.
stop_time	50 ns	Tiempo de simulación específico para este caso. Sobreescrbe <code>stop_time_all</code> solo para este caso.
grade reduction	25 %	Penalización si el caso falla. Se admite porcentaje (%) o valor absoluto. Si se omite, la penalización se reparte equitativamente entre todos los casos.

Secciones de señales

Dentro de cada caso se declaran dos secciones:

- **input:** — valores que se aplican a los puertos de entrada.
- **output:** — valores esperados en los puertos de salida.

Cada señal se especifica en una línea indentada con el formato:

NOMBRE_SEÑAL = valor

El valor puede ser:

- **Escalar** — un único vector de bits: $A = 1010$.
- **Array** — varios valores para múltiples ciclos de reloj: $X = [0, 1, 1, 0]$.
- **Comodín** — guion bajo `_` dentro de un array de salida indica que ese ciclo no se verifica: $Z = [_, _, 1, 0]$.

Si alguna señal de entrada o salida usa sintaxis de array, el evaluador activa el **modo secuencial**: genera un testbench que aplica un valor por ciclo de reloj y verifica la salida al final de cada ciclo. Sin arrays y sin reloj detectado, opera en **modo combinatorio**: aplica las entradas, espera `stop_time` y verifica las salidas una sola vez.

10.1.2 Sintaxis específica para evaluar código VHDL

Circuitos combinatorios

Para circuitos sin reloj se omite la configuración global de clock/reset. El evaluador aplica los valores de `input:`, espera `stop_time` y compara las salidas declaradas en `output:`.

```

1  # Tiempo de espera por defecto para todos los casos
2  stop_time_all = 50 ns
3
4  case = Suma basica sin acarreo
5  stop_time = 20 ns
6  grade reduction = 50%
7  input:
8      A   = 0011
9      B   = 0101
10     Co  = 0
11 output:
12     S   = 1000
13     C4  = 0
14
15 case = Suma con acarreo de entrada
16 stop_time = 20 ns
17 grade reduction = 50%
18 input:
19     A   = 1111

```

```

20     B  = 0001
21     Co = 1
22 output:
23     S  = 0001
24     C4 = 1

```

Listing 10.1: Caso combinatorio — sumador de 4 bits con acarreo*Circuitos secuenciales*

Para circuitos con reloj se declara la configuración global antes del primer `case`. Los valores de entrada y salida se escriben como arrays [`v0`, `v1`, ...], donde cada elemento corresponde a un ciclo de reloj. El evaluador genera el reloj, aplica el reset durante `reset_cycles` ciclos y luego recorre el array ciclo a ciclo.

```

1  clock          = CLK
2  clock_period   = 10 ns
3  reset          = RESET
4  reset_active   = 0          # activo bajo
5  reset_type     = async
6  reset_cycles   = 2
7  stop_time_all  = 1 ms
8
9  # Secuencia X=[0,0,1,1] desde estado inicial C
10 # Transiciones: C->B->A->B->C
11 case = Recorrido C-B-A-B-C
12 grade reduction = 60%
13 input:
14     X = [0, 0, 1, 1]
15 output:
16     Z1 = [1, 0, 1, 1]
17     Z2 = [0, 1, 1, 0]
18
19 case = Una transicion C a B
20 grade reduction = 40%
21 input:
22     X = [0]
23 output:
24     Z1 = [1]
25     Z2 = [0]

```

Listing 10.2: Caso secuencial — FSM Mealy**Longitud de arrays**

Los arrays de `input:` y `output:` de un mismo caso deben tener la misma longitud. El evaluador no rellena valores faltantes.

Uso del comodín _

El comodín `_` en un array de salida indica que ese ciclo no se verifica. Es útil cuando la salida del circuito no está definida durante los primeros ciclos (por ejemplo, mientras se llena un pipeline o un registro de desplazamiento).

```
1  clock          = CLK
2  clock_period   = 10 ns
3  reset          = CLR
4  reset_active   = 1
5  reset_type     = async
6  reset_cycles   = 2
7  stop_time_all  = 1 ms
8
9  # Ciclo 1: carga L=1 -> Q=1000000000000
10 # Ciclo 2: desplaza SHE=1 -> Q=0100000000000
11 # Ciclo 3: desplaza SHE=1 -> Q=0010000000000
12 case = Cargar y desplazar
13 grade reduction = 35%
14 input:
15     L    = [1, 0, 0]
16     SHE  = [0, 1, 1]
17     ES   = [0, 0, 0]
18     D    = 1000000000000
19 output:
20     LED  = [_ , 010001000000000000 , 010000100000000000]
```

Listing 10.3: Comodin en salidas de registro de desplazamiento

10.2 Usar el archivo de casos para evaluar diseños VHDL

Una vez que se ha mencionado la sintaxis general de los casos de prueba, se pueden escribir casos específicos para evaluar diseños VHDL. Para ello, usaremos el archivo `vpl_evaluate.cases` que se encuentra dentro de los archivos de ejecución o en la sección de configuración de test de la actividad VPL, y se pueden escribir casos específicos para evaluar los diseños VHDL entregados por los alumnos.

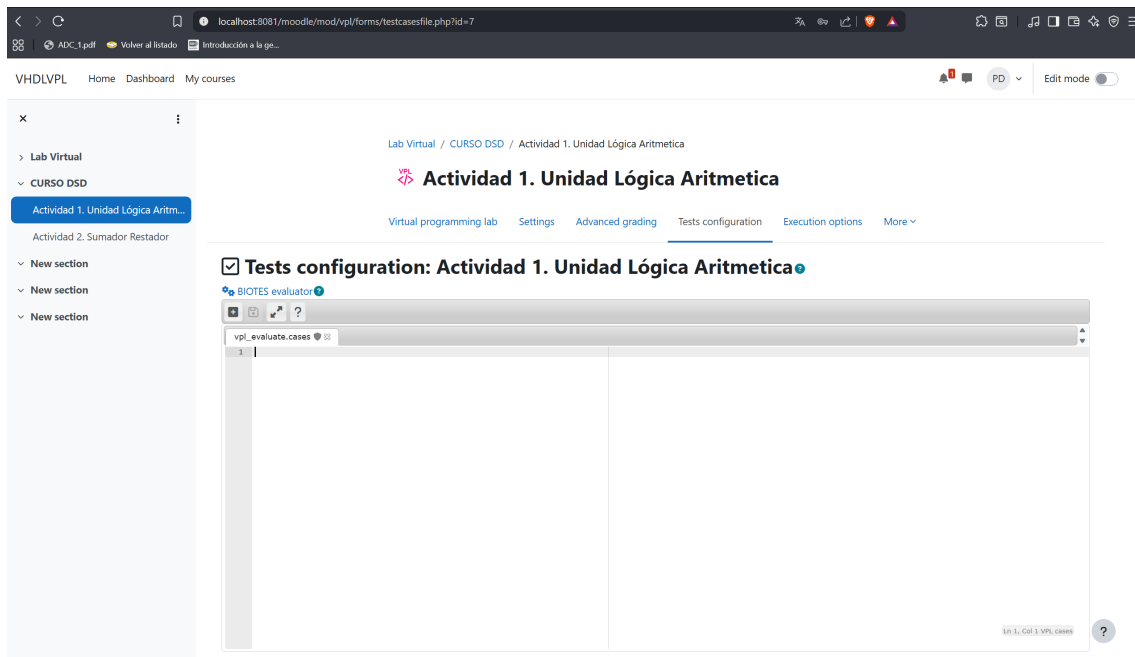


Figura 10.1: Archivo de configuración de casos de prueba en la actividad VPL

La edición de este archivo también se hace usando el editor de código integrado en VPL, por lo que se pueden usar las mismas herramientas de edición, como el resaltado de sintaxis, la indentación automática y la búsqueda de texto para facilitar la escritura de los casos de prueba. Una vez escritos los casos de prueba, se deben guardar los cambios para que el evaluador pueda usarlos al calificar los diseños entregados por los alumnos.

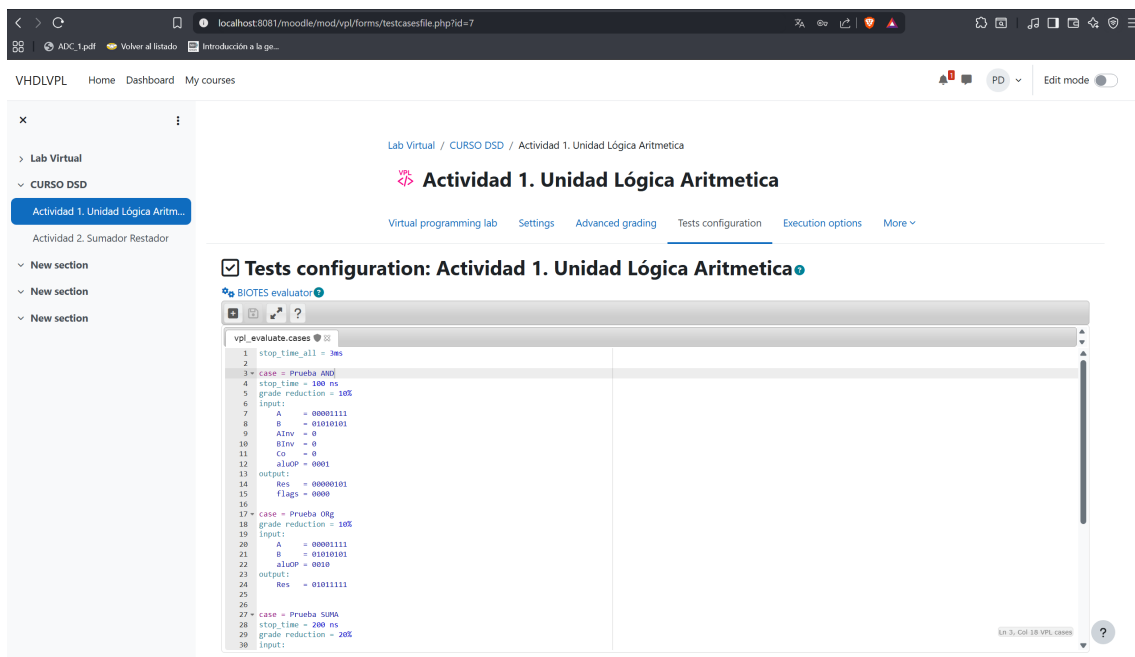


Figura 10.2: Archivo de casos de prueba creado y guardado en la actividad VPL

Una vez teniendo los casos de prueba escritos y guardados, el evaluador los

ejecutará automáticamente cada vez que un alumno envíe su diseño para evaluación, generando un reporte con los resultados de cada caso y la calificación final basada en las penalizaciones definidas. En la sección [11](#) se explica cómo el profesor puede verificar que los casos de prueba funcionan correctamente antes de que los alumnos entreguen sus diseños.

Test de Evaluación por Parte del Profesor

VPL permite al profesor verificar que el sistema de evaluación automática funciona correctamente *antes* de que los alumnos entreguen sus soluciones. Para ello, el profesor proporciona soluciones de referencia —correctas e incorrectas— y VPL las evalúa automáticamente, generando un reporte que indica si cada caso obtuvo la calificación esperada.

11.1 Propósito

El test de evaluación sirve para:

- Confirmar que los casos de prueba (`vpl_evaluate.cases`) detectan errores en diseños incorrectos.
- Verificar que una solución correcta obtiene la calificación máxima.
- Detectar errores de configuración en los scripts de evaluación antes de publicar la actividad.
- Repetir la verificación cada vez que se actualicen los scripts o se cambie el servidor de ejecución.

11.2 Estructura de la carpeta de pruebas

Dentro de los **Archivos de ejecución** de la actividad se debe crear la carpeta raíz:

```
vpl_evaluation_tests/
```

Cada subcarpeta dentro de ella representa un caso de prueba y debe seguir *exactamente* el formato:

```
[pass|fail]-<nombre> [-<calificacion>]
```

- **pass** — solución correcta; se espera calificación igual a la máxima definida en la actividad (o igual a *<calificacion>* si se especifica).
- **fail** — solución incorrecta; se espera calificación *menor* que la máxima (o exactamente *<calificacion>* si se especifica).
- *<nombre>* — descripción breve del escenario, sin guiones.
- *<calificacion>* — opcional; calificación numérica esperada (entero o decimal).

Cada subcarpeta debe contener los archivos fuente del diseño que se desea evaluar (por ejemplo, el archivo `.vhd` del circuito).

11.2.1 Ejemplos de nombres de subcarpeta

Nombre	Significado
pass-Solucion correcta-10	Diseño correcto; se espera nota 10
pass-Correcto	Diseño correcto; se espera nota máxima de la actividad
fail-Error de salida-3	Diseño incorrecto; se espera nota exacta 3
fail-Ciclo infinito-0	Diseño con bucle infinito; se espera nota 0
fail-Error de compilacion	Diseño que no compila; se espera nota < máxima

11.2.2 Estructura de directorios

La jerarquía de archivos debe quedar así:

```
vpl_evaluation_tests/
  pass-Solucion correcta-10/
    mi_circuito.vhd
  fail-Error de salida-3/
    mi_circuito.vhd
  fail-Ciclo infinito-0/
    mi_circuito.vhd
  fail-Error de compilacion/
    mi_circuito.vhd
```

Listing 11.1: Estructura mínima de `vpl_evaluation_tests`

Requisito mínimo

Debe existir **al menos una subcarpeta** de tipo **pass**. Sin ella, VPL reporta un error de configuración y no ejecuta ninguna prueba.

11.2.3 Actividades con variaciones

Si la actividad usa variaciones, se crea una subcarpeta por variación dentro de `vpl_evaluation_tests`. Cada subcarpeta de variación contiene a su vez las subcarpetas `pass-` y `fail-` correspondientes.

```
vpl_evaluation_tests/
  variacion1/
    pass-Correcto-10/
      mi_circuito.vhd
    fail-Salida erronea-0/
      mi_circuito.vhd
  variacion2/
    pass-Correcto-10/
      mi_circuito.vhd
    fail-Salida erronea-0/
      mi_circuito.vhd
```

Listing 11.2: Estructura con variaciones

11.3 Pasos para ejecutar el test

Paso 1

Acceder a la actividad VPL en Moodle y abrir la página **Archivos de ejecución** (menú de administración de la actividad).

Paso 2

Crear la carpeta `vpl_evaluation_tests` y las subcarpetas de casos de prueba con sus archivos fuente, siguiendo el formato descrito en la sección 11.2.

Paso 3

Desde la misma página de **Archivos de ejecución**, hacer clic en el botón **Evaluar**. VPL ejecutará el proceso de evaluación usando los casos definidos en `vpl_evaluation_tests`.

Paso 4

Revisar el reporte generado (véase sección 11.4). Si alguna prueba falla, corregir los casos de prueba o los scripts y repetir desde el paso 3.

Seguridad

VPL elimina automáticamente la carpeta `vpl_evaluation_tests` cuando la actividad se encuentra fuera del modo de evaluación del profesor. Esto impide que los alumnos accedan a las soluciones de referencia.

11.4 Reporte de evaluación

Al ejecutar el test, VPL genera un reporte en el panel de **Comments** con tres bloques diferenciados: reporte de compilación, reporte de evaluación resumido y reporte de evaluación completo (*Full Evaluation Report*).

11.4.1 Configuración previa de los casos de prueba

Antes de ejecutar el test es necesario que el archivo `vpl_evaluate.cases` contenga los casos definidos correctamente. Como se describió en el capítulo 10, este archivo se edita desde la pestaña **Tests configuration** de la actividad VPL (véase Figura 10.1). La Figura 10.2 muestra el archivo con los casos ya escritos y guardados, listos para ser ejecutados por el evaluador automático.

11.4.2 Reporte de compilación

Indica si el evaluador pudo preparar la solución de referencia para cada caso de prueba. VPL muestra los pasos realizados: detección del lenguaje de programación, carga del archivo fuente y verificación del tipo de caso (`pass` o `fail`).

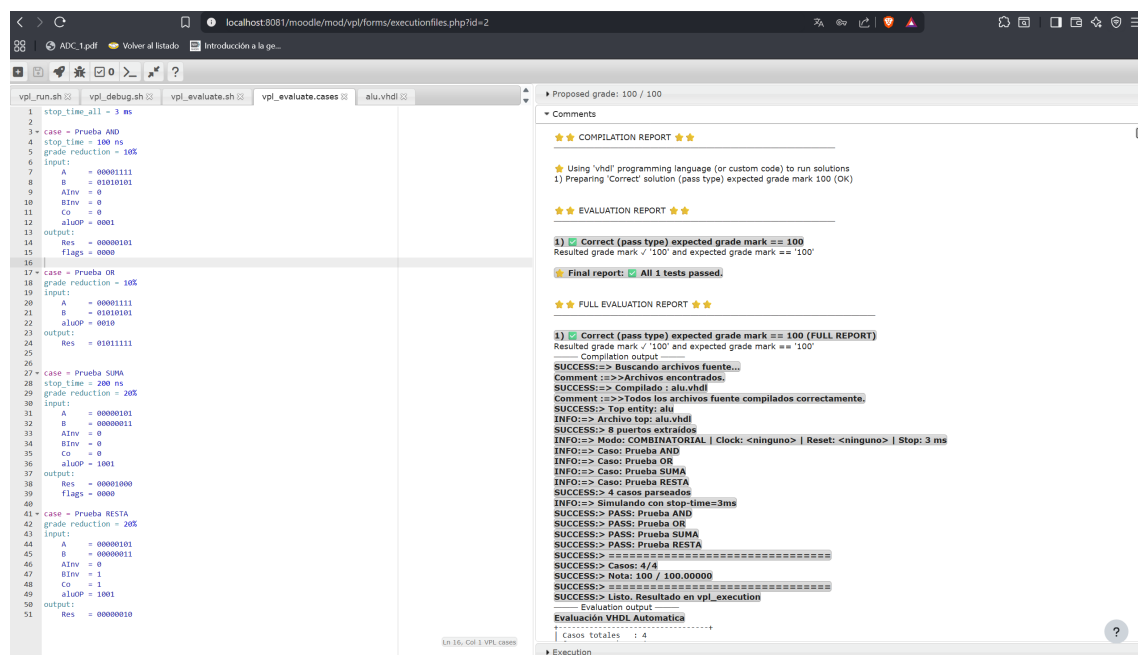


Figura 11.1: Reporte generado por VPL al evaluar todos los casos de prueba correctamente

La Figura 11.1 muestra el panel de **Comments** con los tres bloques del reporte tras ejecutar la evaluación con una solución correcta. La sección **COMPILATION REPORT** confirma que los archivos fuente fueron compilados sin errores y que la solución de tipo `pass` con calificación esperada de 100 fue preparada correctamente.

11.4.3 Reporte de evaluación (resumen)

Muestra para cada caso de `vpl_evaluation_tests` si la calificación obtenida coincide con la esperada. El formato de cada entrada es:

```
<n>) <tipo>(pass|fail type) expected grade mark == <esperada>
Resulted grade mark = <obtenida>and expected grade mark == <esperada>
```

Al final aparece la línea **Final report** que resume el resultado global:

Final report: *checkmark* All <n>tests passed.

11.4.4 Reporte de evaluación completo (*Full Evaluation Report*)

El bloque **FULL EVALUATION REPORT** detalla la ejecución interna del evaluador para cada caso. En él se pueden identificar:

- **Compilation output** — resultado de compilar el diseño con GHDL.
- Líneas **SUCCESS:=>** — operaciones completadas correctamente: búsqueda de archivos, detección de entidad top, extracción de puertos y simulación.
- Líneas **INFO:=>** — configuración detectada: modo combinatorio o secuencial, reloj, reset y casos cargados.
- Líneas **PASS: / FAIL:** — resultado individual de cada caso de prueba.
- **Nota final** — calificación calculada en formato **Nota: <n>/ 100.0000**.
- **Evaluation output** — resumen estadístico: número de casos totales, correctos y fallidos.

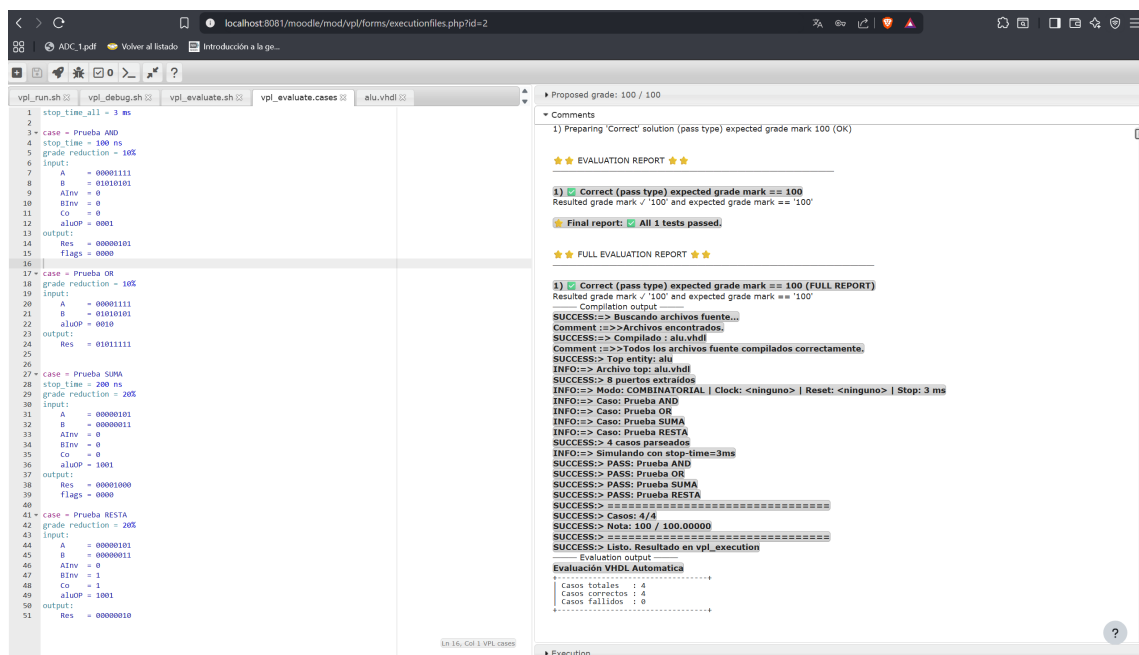


Figura 11.2: Detalle del *Evaluation output* y resumen estadístico al final del reporte

La Figura 11.2 muestra el *Evaluation output* al final del reporte completo, indicando **casos totales**, **casos correctos** y **casos fallidos**. Esta sección es la que el alumno recibe como comentario de retroalimentación cuando VPL evalúa su entrega.

11.4.5 Reporte con casos fallidos

Cuando un caso de prueba no produce la calificación esperada, el reporte lo marca como fallo. La Figura 11.3 ilustra un escenario donde el evaluador detecta un diseño incorrecto: aparece la línea **ERROR:=>FAIL: <caso>** en el reporte completo, indicando exactamente qué caso no pasó. Esto permite al profesor verificar que los casos de prueba detectan correctamente los errores en los diseños de los alumnos.

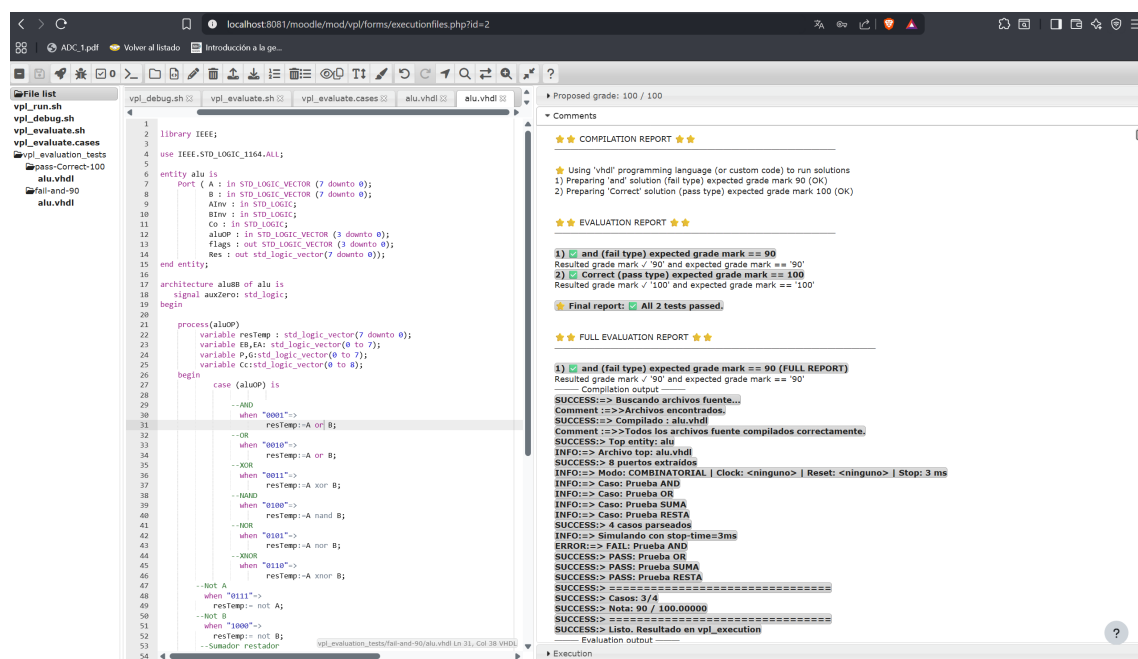


Figura 11.3: Reporte de evaluación con casos fallidos detectados por el evaluador automático

Interpretación de casos fail

Un caso de tipo fail en `vpl_evaluation_tests` *pasa* el test cuando el evaluador asigna al diseño incorrecto una calificación menor que la máxima. Si el reporte muestra **Final report: All <n>tests passed** para un conjunto que incluye casos fail, significa que los casos de prueba detectan correctamente los diseños erróneos.



Glosario de Términos

B

Referencia Rápida de Comandos

Recursos y Referencias
